

Ch 6: Dynamic Programming and Greedy Algorithms

Data Structure and Algorithm Design

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal
Former Guest Faculty, Darmstadt University of Applied Sciences, Germany

CamScanner



Overview

Ch 6: Dynamic Programming and Greedy Algorithms

- Engineering applications of dynamic programming: Resource Scheduling
- Greedy algorithms: Huffman coding and its relevance to data compression in embedded systems
- Optimizing dynamic programming techniques for real-time systems

CamScanner

1. Introduction to Dynamic Programming
2. Optimizing Dynamic Programming
3. Application of Dynamic Programming: Resource Scheduling

DYNAMIC PROGRAMMING

CamScanner

Dynamic Programming

- DP is general and powerful algorithm design technique
- **Idea**:
  - Divide the problem → **subproblems**
  - Find the **solutions** s_i of those subproblems
  - **Reuse** s_i (avoids the redundant computations)
- It is a powerful optimization technique



CamScanner

Dynamic Programming

- How to ***reuse*** the solutions of subproblems?
  - **Memoization**
    - Applied when you use *recursive* solution
    - Top-down approach
  - **Tabulation**
    - Applied when you use *iterative* solution
    - Bottom-up approach

- Which is one is better- memoization or Tabulation?



CamScanner

Fibonacci Numbers

- $F = \{1,1,2,3,5,8,13,21,34,55\}$
- Where $F_1 = 1, F_2 = 1$
- $\therefore F_n = F_{n-1} + F_{n-1}$

$$F_n = \begin{cases} 1 & \text{if } n = 1 \,||\, n = 2 \\ F_{n-1} + F_{n-2} & \text{if } n > 2 \end{cases}$$

- Goal: compute F_n

CamScanner

Naive Recursive Solution

- ***Goal***: compute F_n

$$F_n = \begin{cases} 1 & \text{if } n = 1 \,||\, n = 2 \\ F_{n-1} + F_{n-2} & \text{if } n > 2 \end{cases}$$

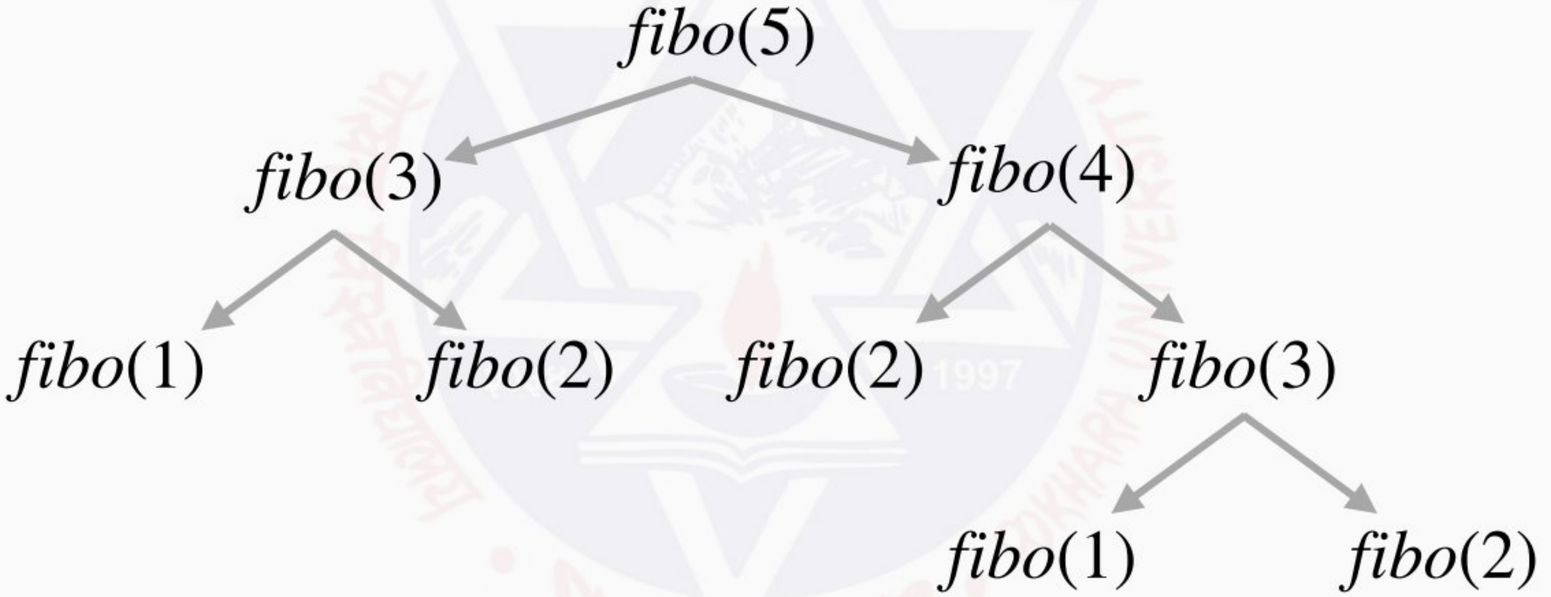
```
int fibo(int n) {
    if (n<=2)
      return 1;
    else {
      return fibo(n-2) + fibo(n-1);
    }
}
```



CamScanner

Naive Recursive Solution

- ***Goal***: compute $fibo(n)$ for $n = 5$

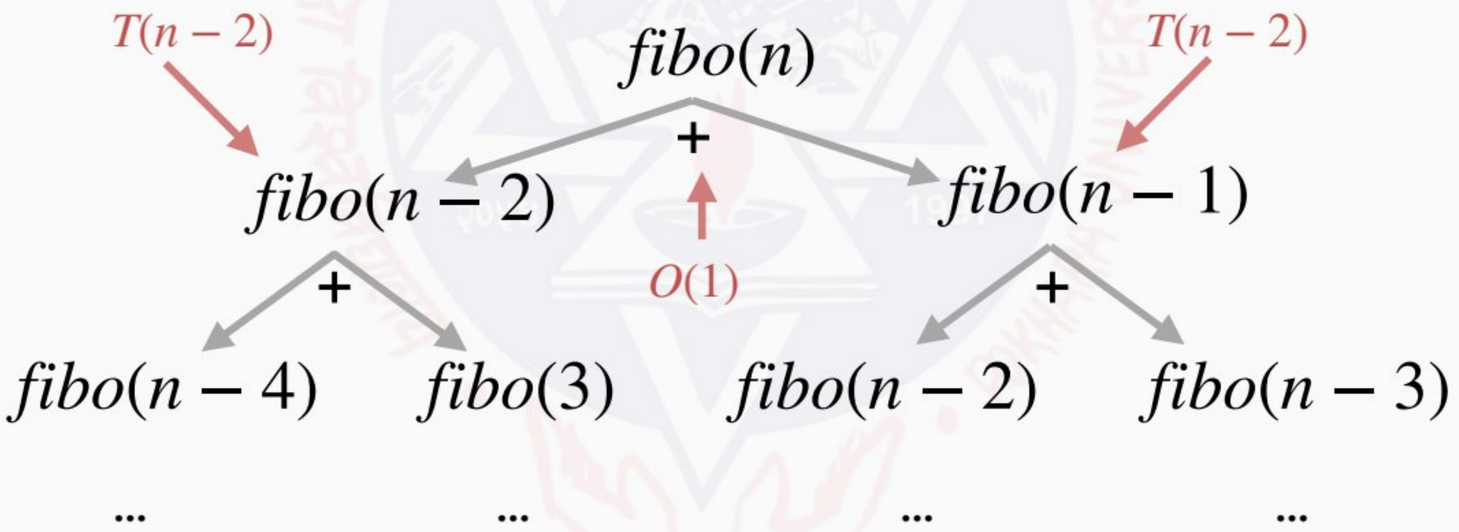


- The naive recursive solution computes all these sub-problems.

Naive Recursive Solution

- Total Running time is **exponential**

$$T(n) = T(n-2) + T(n-1) + O(1)$$



Naive Recursive Solution

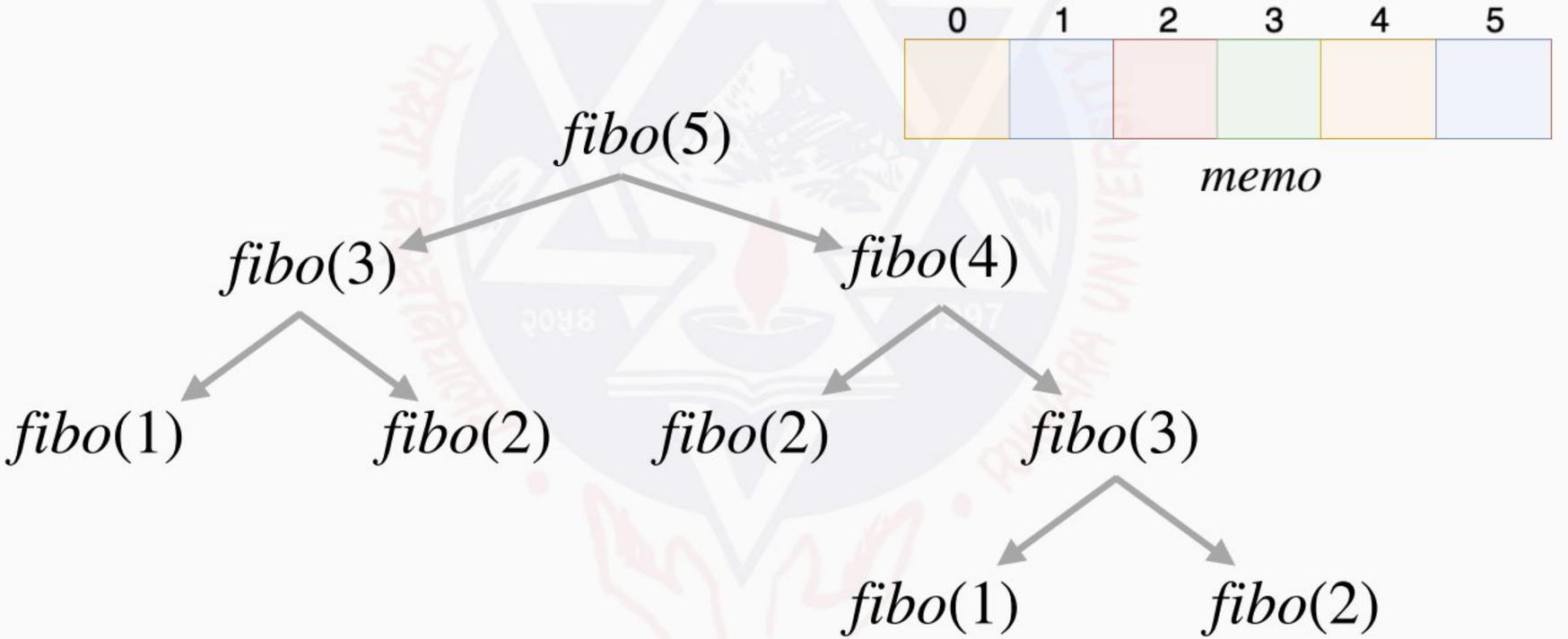
- Total Running time is **exponential**

$$
\begin{aligned}
T(n) &= T(n-2) + T(n-1) + O(1) \\
&= 2T(n-2) + O(1) \\
&= 2(2T(n-2) + O(1)) + O(1) = 4T(n-2) + 3 * O(1) \\
&\ldots \\
&= 2^k T(n-k) + k * O(1) \\
&\ldots. \\
&= 2^n T(0) + n * O(1) = 2^n + n \\
\therefore T(n) &= O(2^n)
\end{aligned}
$$

CamScanner

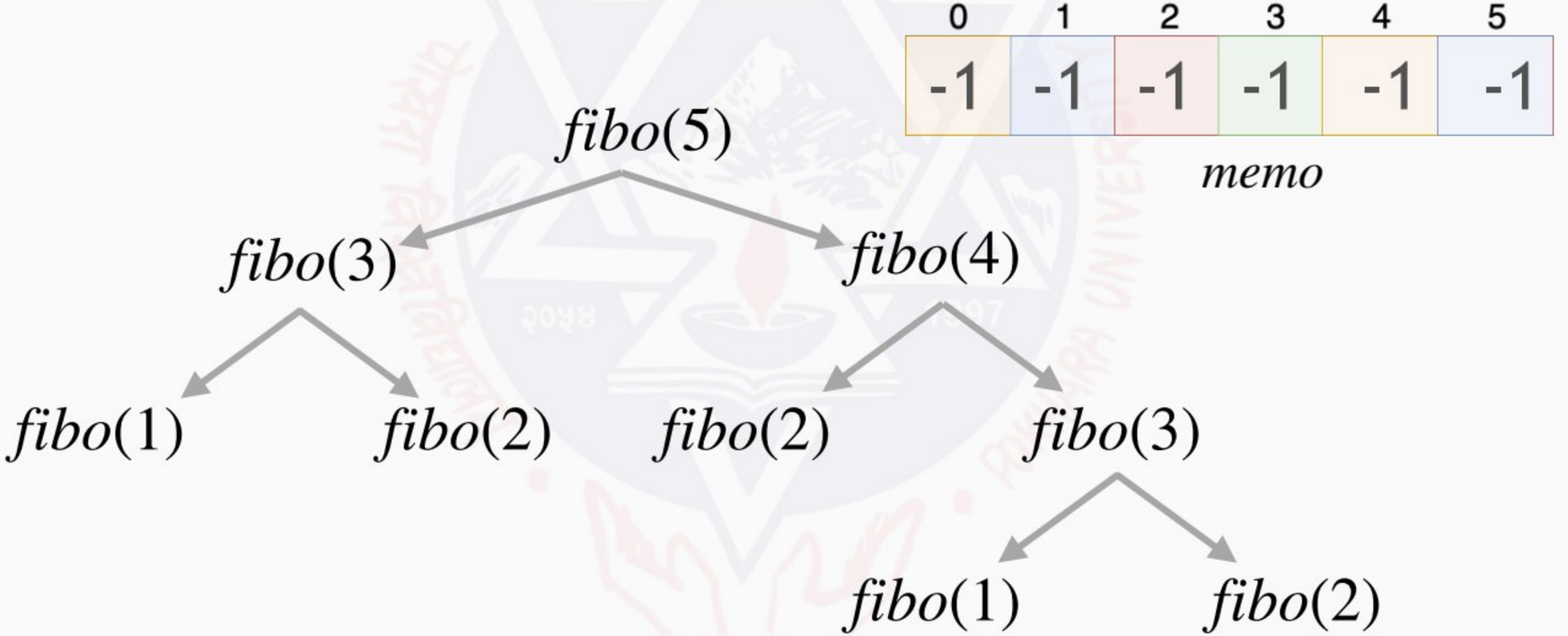
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



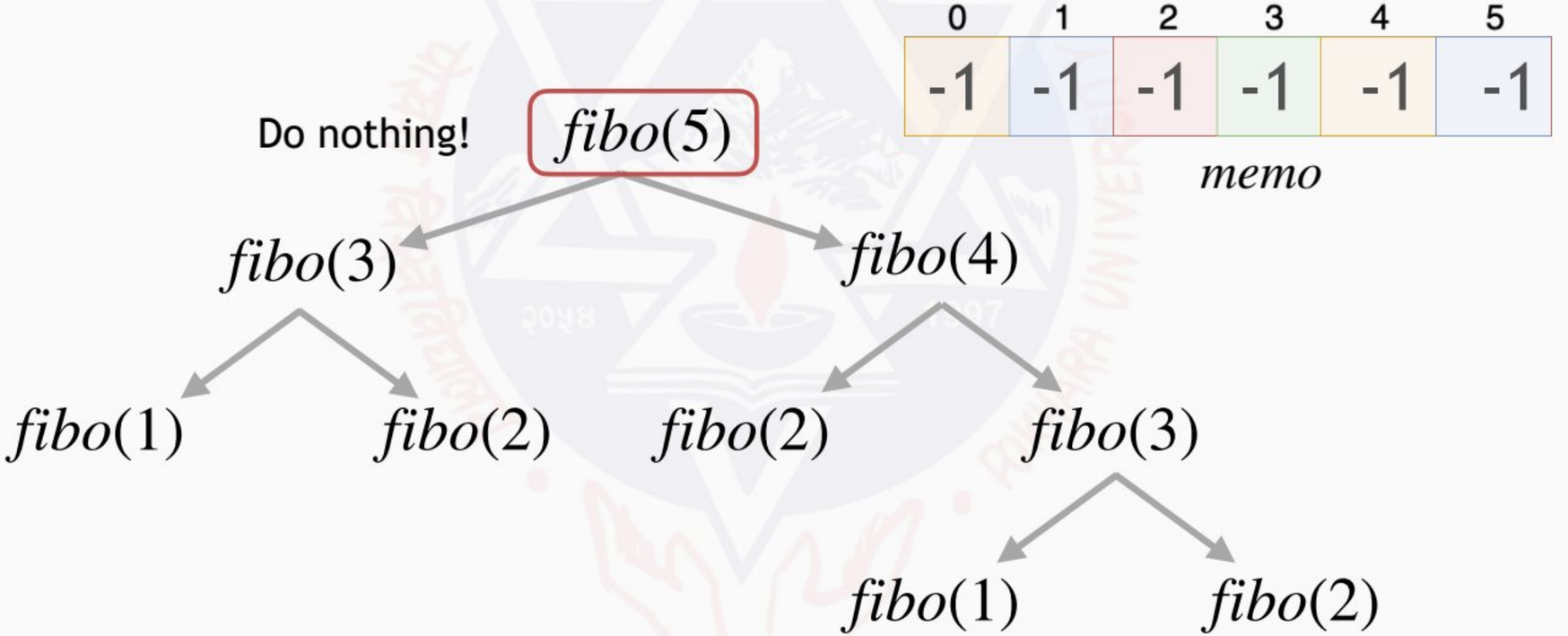
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



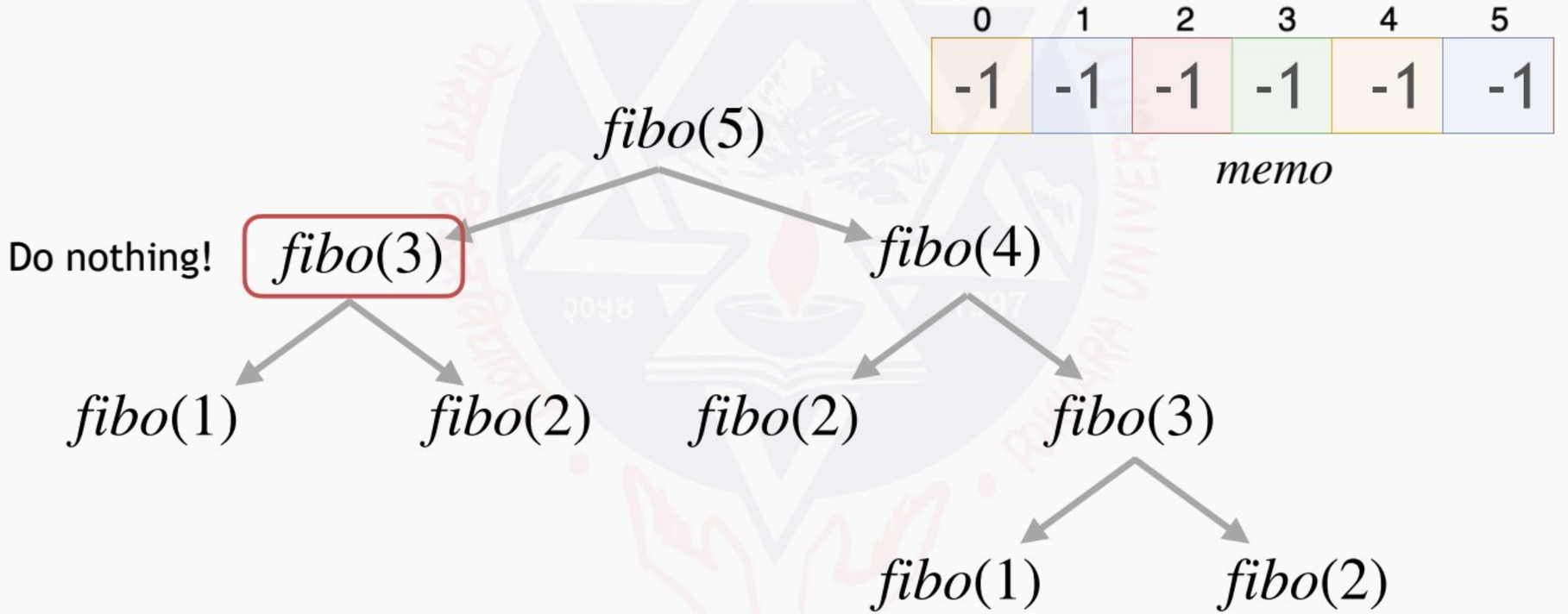
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



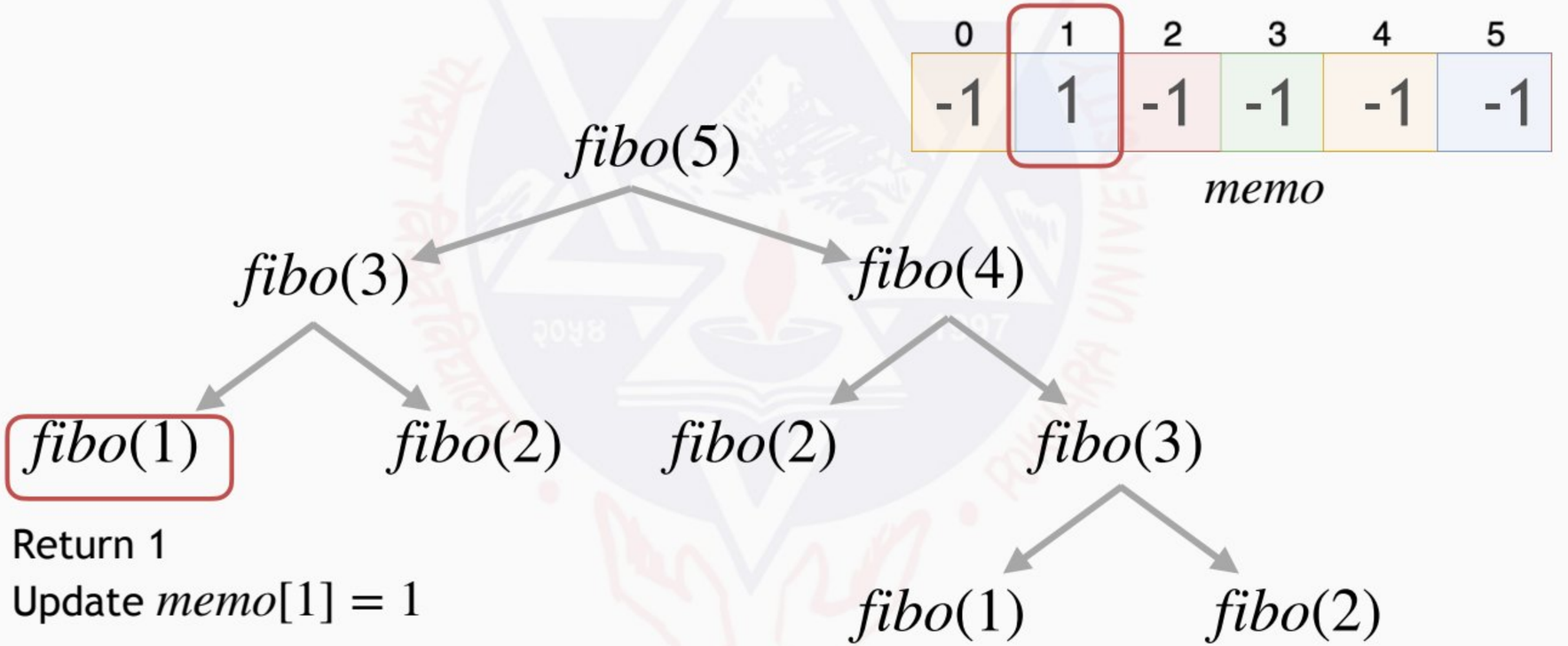
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



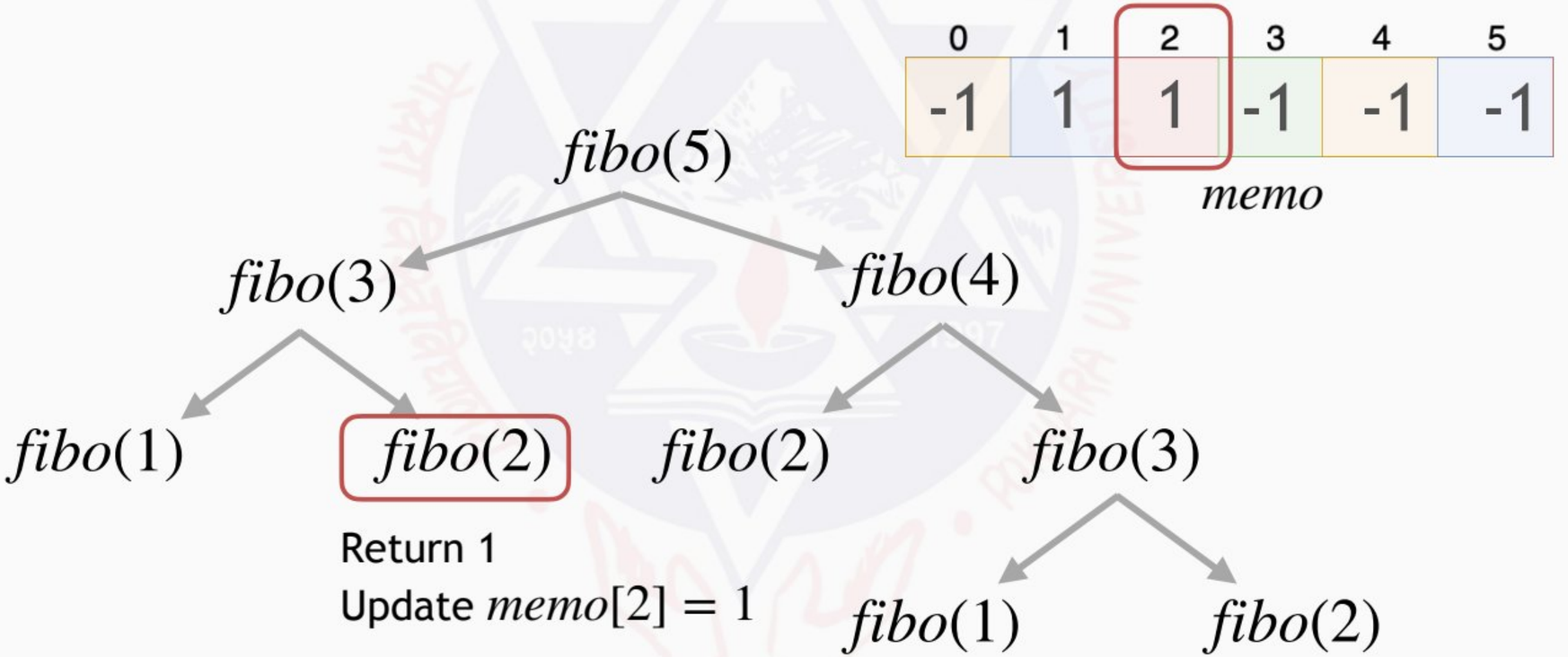
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



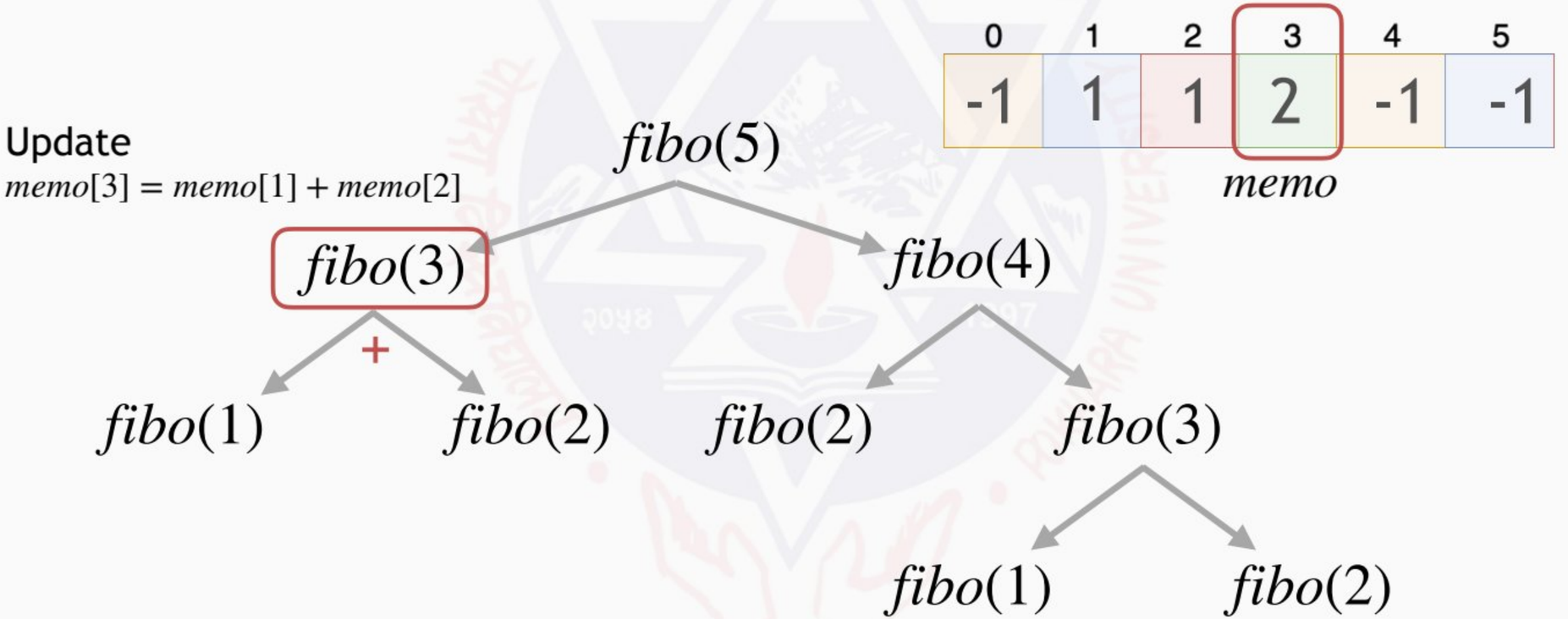
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



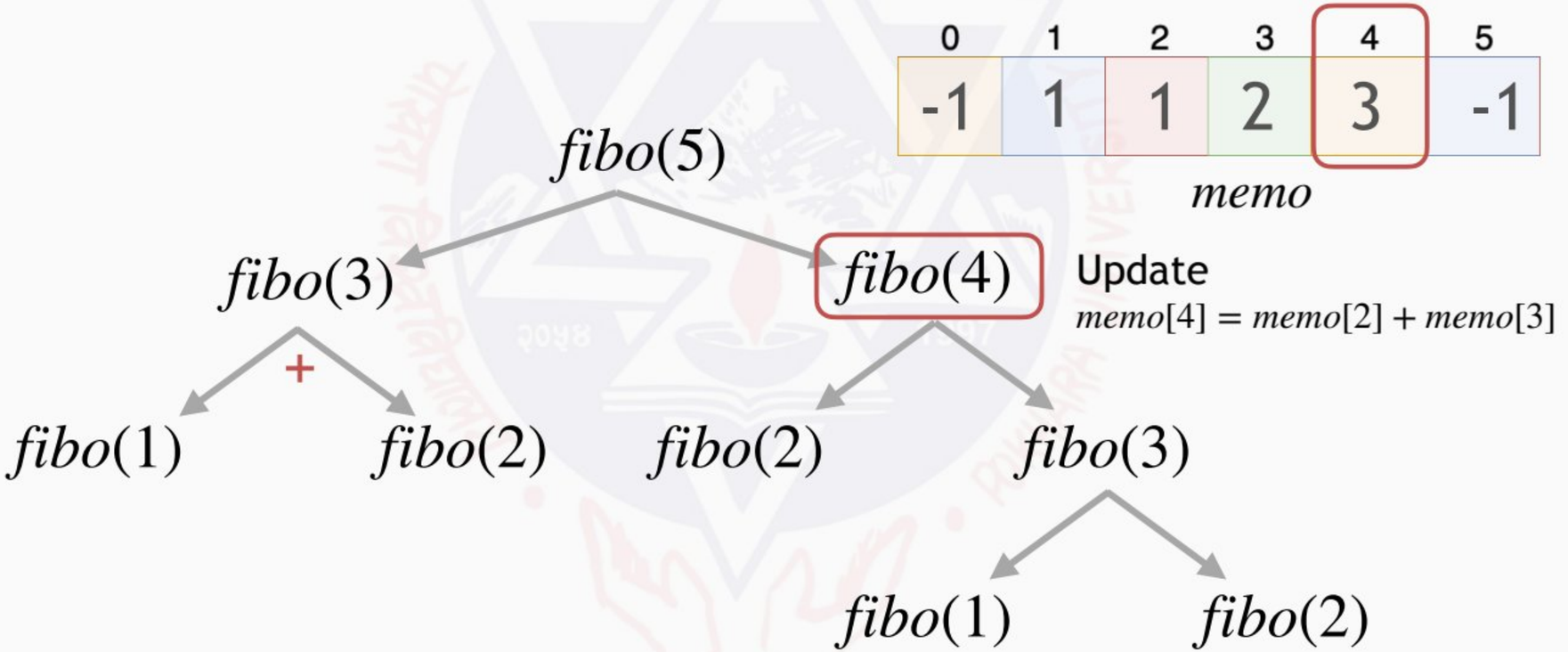
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



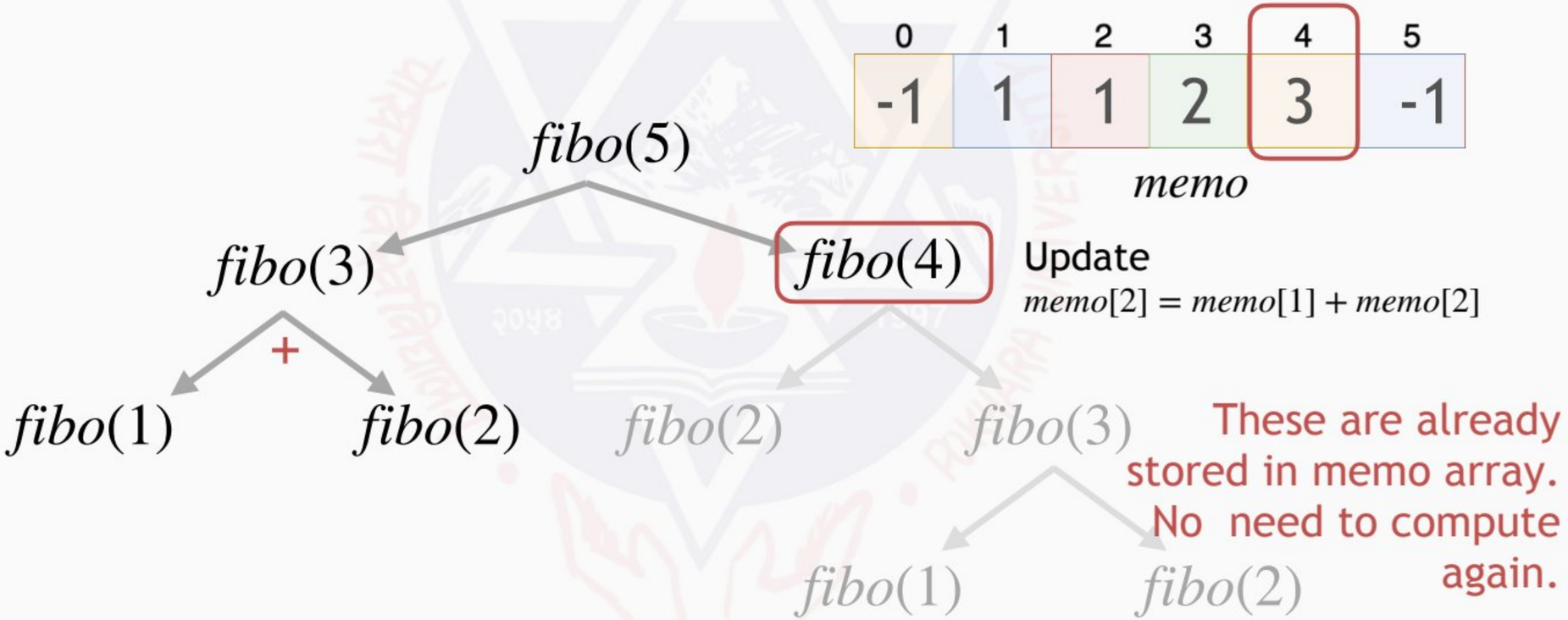
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



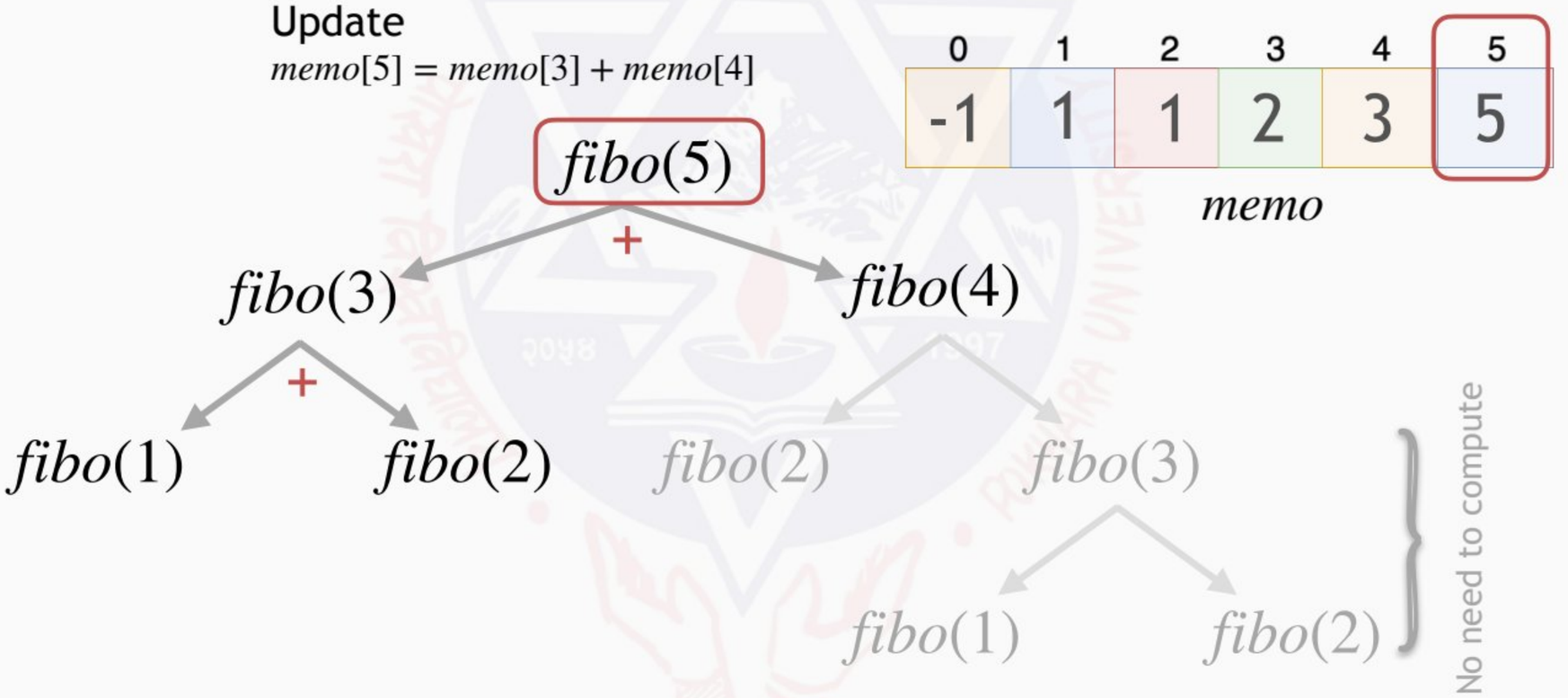
Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ for $n = 5$ using memoization



Memoization of fibo(n)

- ***Goal***: compute $fibo(n)$ using memoization

```
int fibo(int n) {
    // if the value is already calculated, return it
    if (memo[n] != -1) return memo[n];
    if (n<=2)
      return 1;
    else {
        // store the value (nth fibo) in the memo
        memo[n] = fibo(n-2) + fibo(n-1);
    }
    return memo[n];
}
```



CamScanner

Analysis of Memoized of fibo(n)

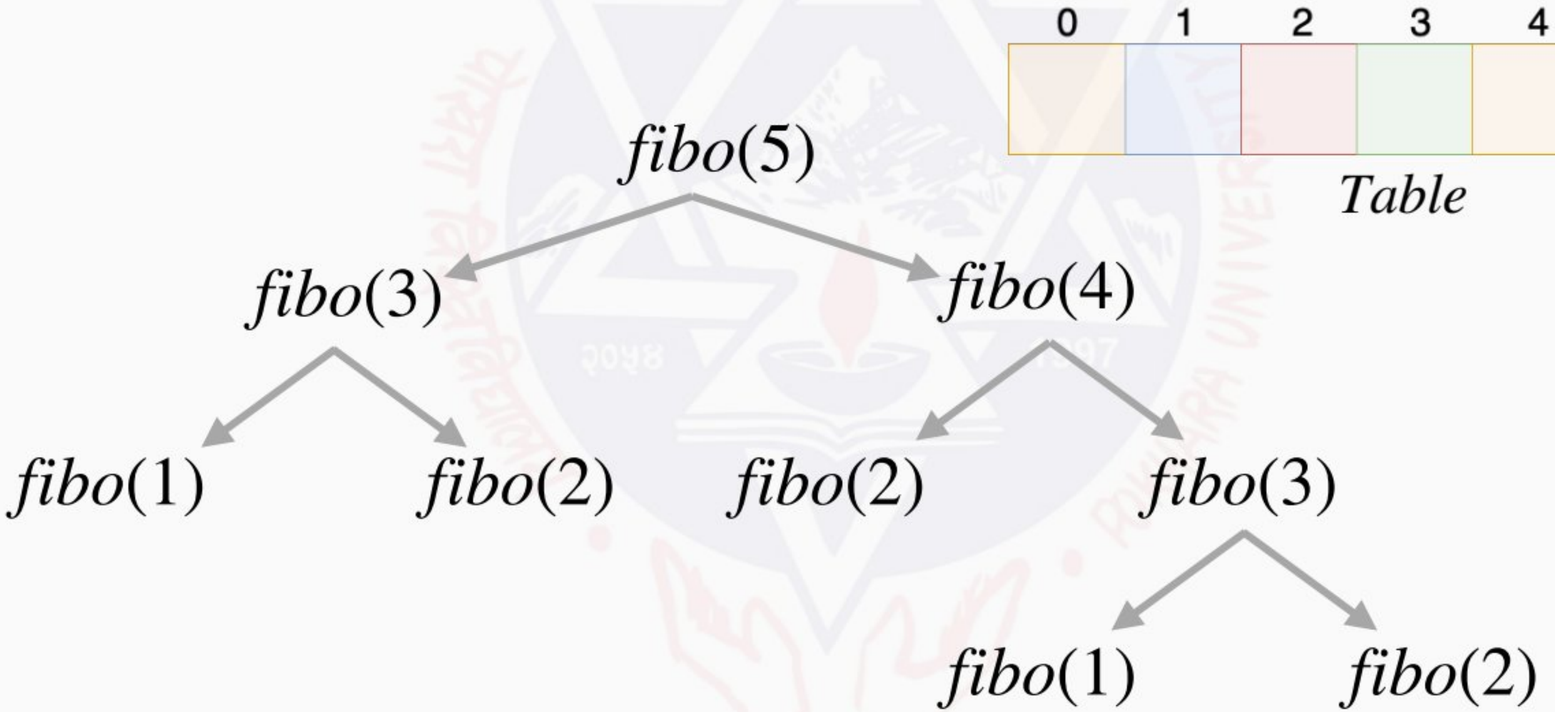
- $fib(n),\ \forall n$, only recurses the first time it is called
  - During this phase, puts the solutions of all sub-problems
  - Once its finds all solutions $\forall n$, it uses the stored sub-solution, combine them, generate the final solution and return it.
- 2 version of recursions
  - Recursion without using memo
    - # of non-memoized call is n - $fibo(1), fibo(2), \ldots, fibo(n)$
    - Non-memoized call cost = $O(1)$
  - Recursion using memo
    - Memoized call cost = $\Theta(1)$
- $\therefore$ Total Running Time, $\boxed{T(n) = O(n)}$



CamScanner

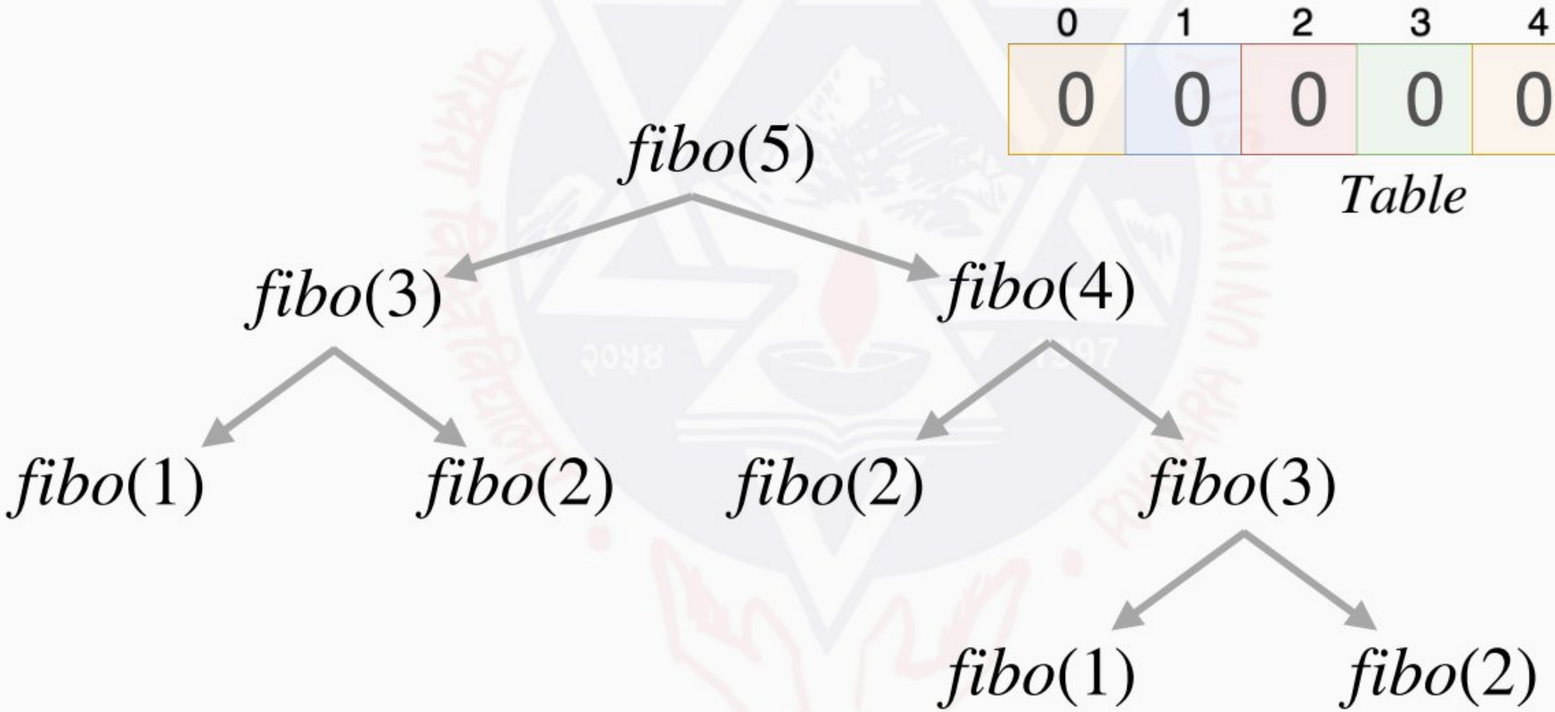
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm



Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm



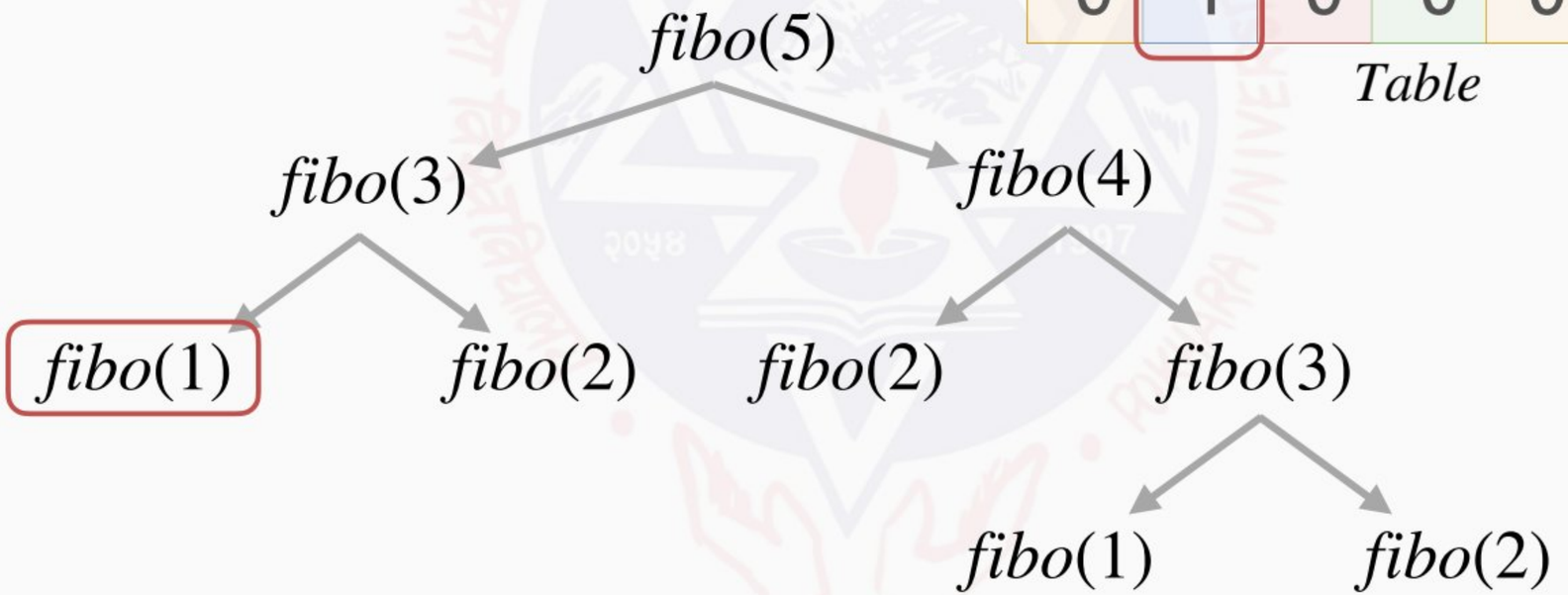
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

Initialize $Table[1] = 1$

0	1	2	3	4	5
0	1	0	0	0	0

Table



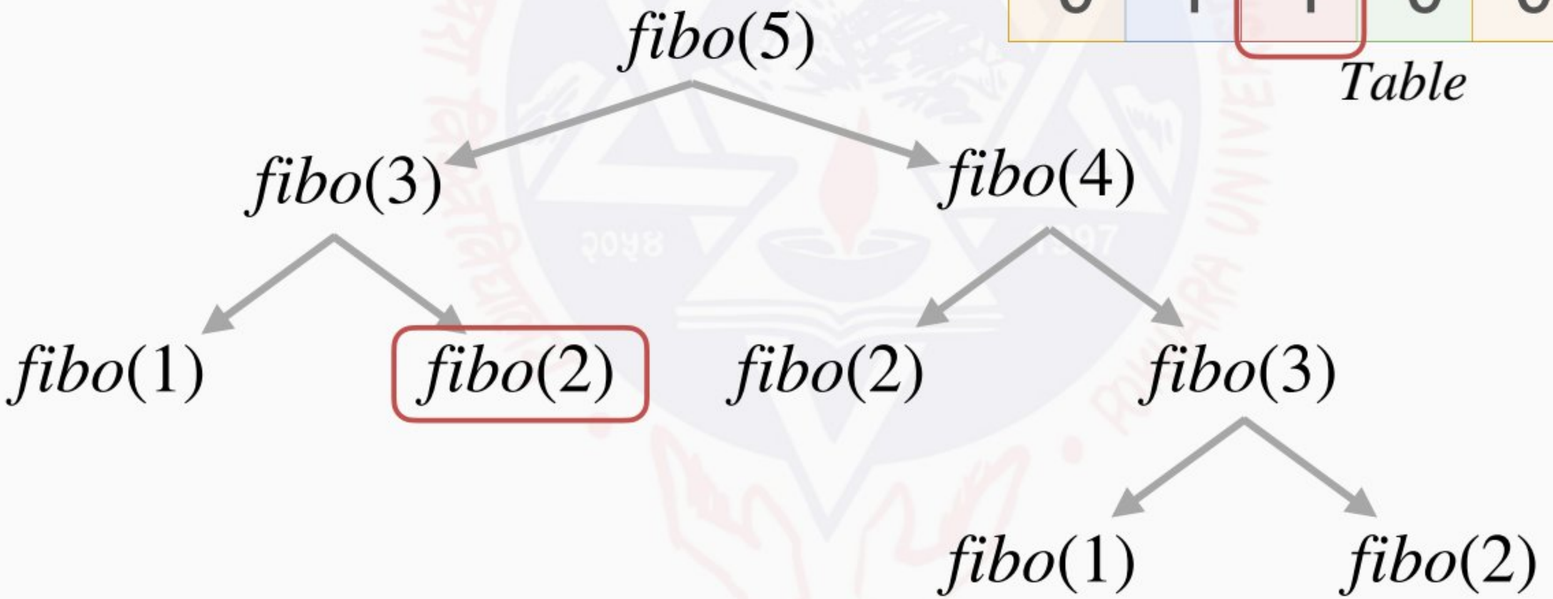
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

$$Table[2] = Table[0] + Table[1]$$

0	1	2	3	4	5
0	1	1	0	0	0

Table



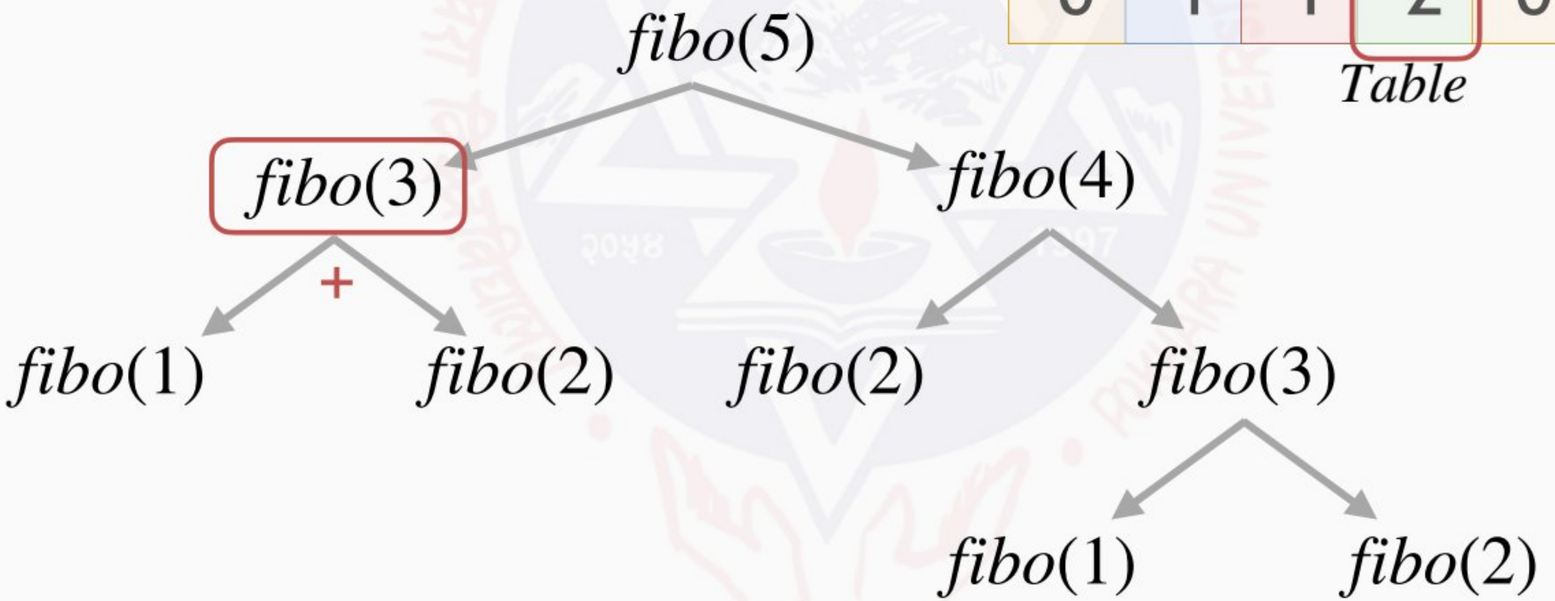
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

$$Table[3] = Table[1] + Table[2]$$

0	1	2	3	4	5
0	1	1	2	0	0

Table



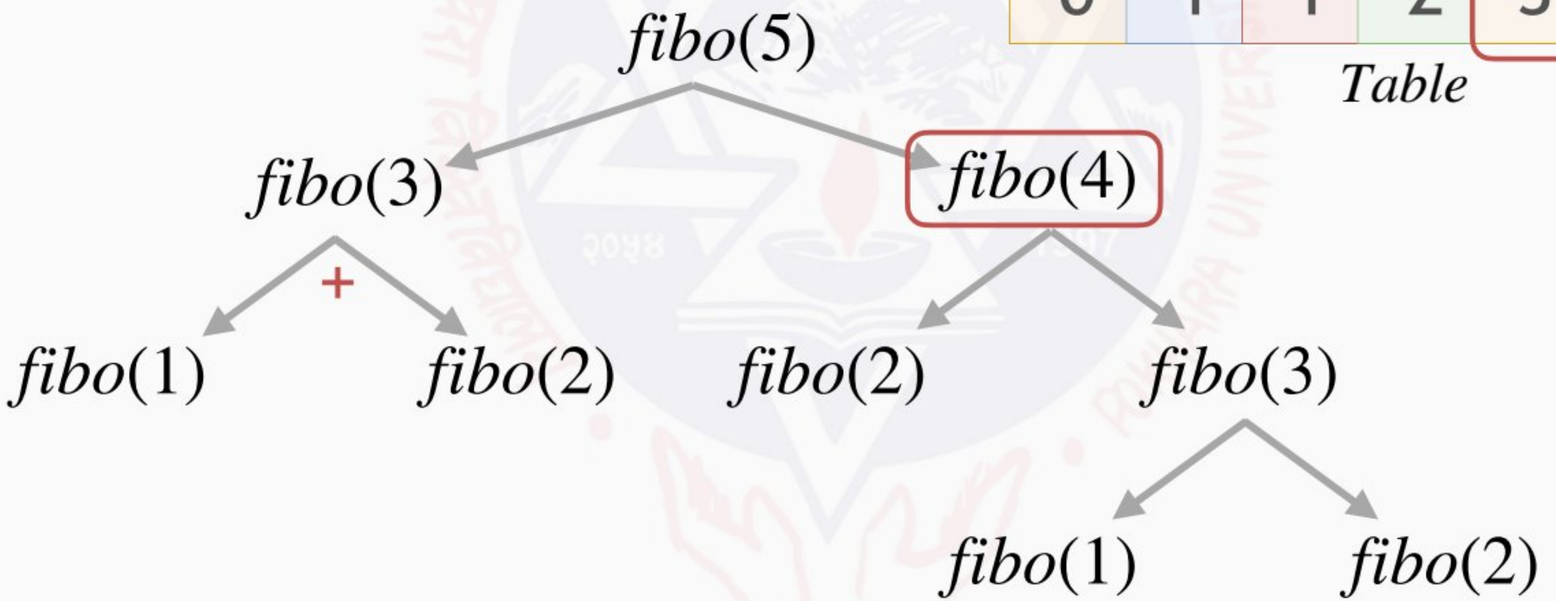
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

$$Table[4] = Table[2] + Table[3]$$

0	1	2	3	4	5
0	1	1	2	3	0

Table



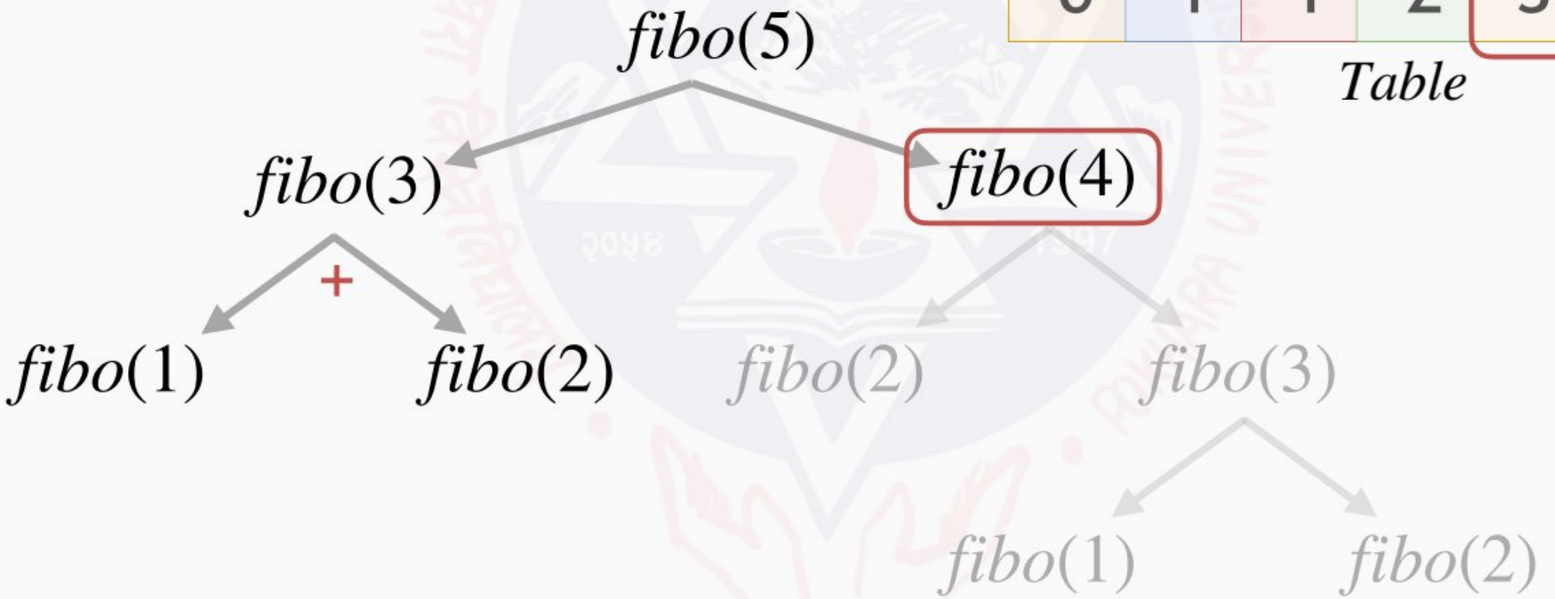
Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

$$Table[4] = Table[2] + Table[3]$$

0	1	2	3	4	5
0	1	1	2	3	0

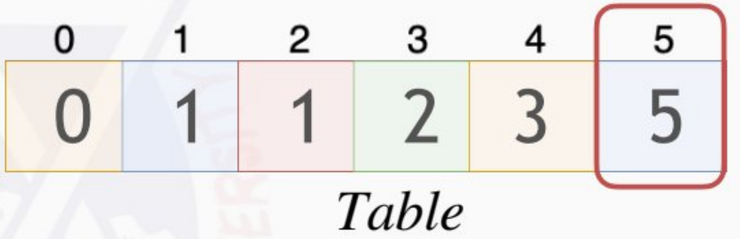
Table

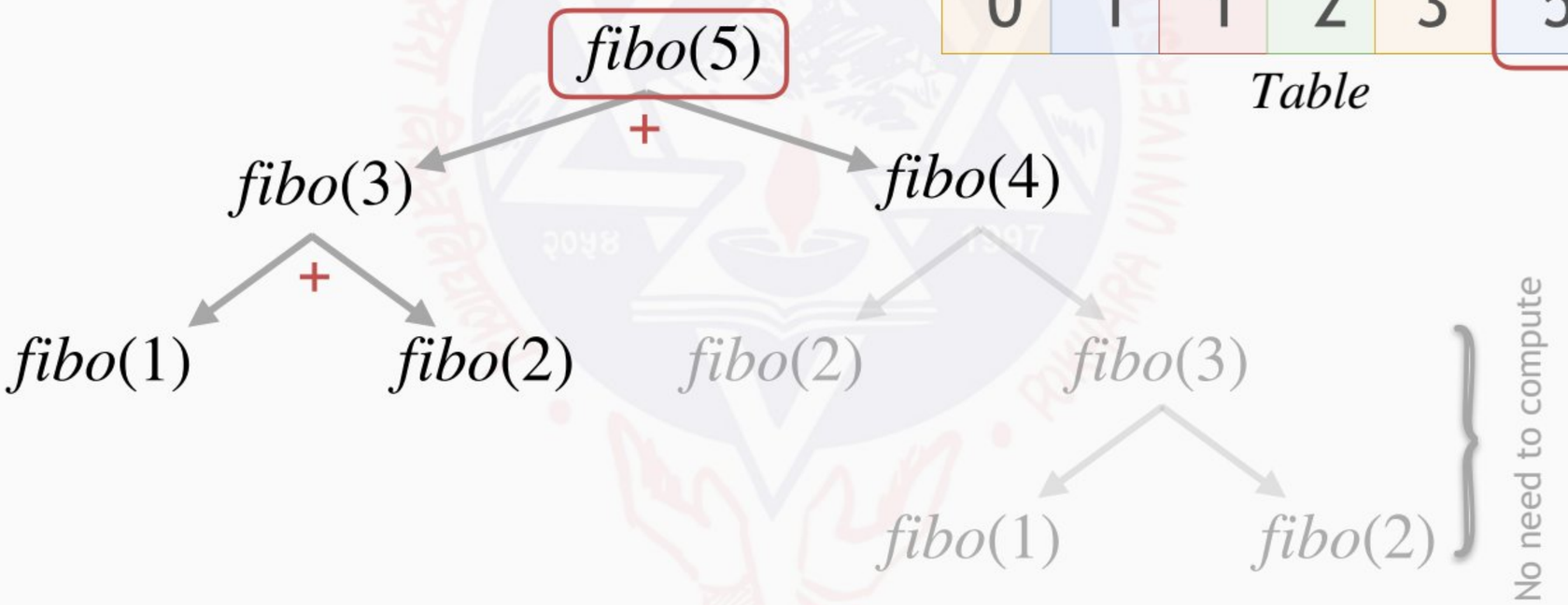


Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ for $n = 5$ using Tabulation DP Algorithm

$$Table[5] = Table[3] + Table[4]$$





Tabulated $fibo(n)$ [Bottom-up DP Algorithm]

- ***Goal***: compute $fibo(n)$ using Tabulation DP Algorithm

```
int fibo(int n) {
    if(n<=2) return 1;
    std::vector<int> Table(n+1, 0);
    Table[1] = 1;
    for (int i=2; i<=n; i++) {
        Table[i] = Table[i-2] + Table[i-1];
    }
    return Table[n];
}
```



CamScanner

Memoized vs Tabulated $fibo(n)$

Memoized

- *Dynamic Programming*
- $T(n) = O(n)$
- *Space* = $O(n)$

Tabulated

- *Dynamic Programming*
- $T(n) = O(n)$
- *Space* = $O(n)$

Which one do you prefer? Why?



CamScanner

Memoized vs Tabulated $fibo(n)$

Memoized

- *Dynamic Programming*
- $T(n) = O(n)$
- *Space* = $O(n)$
- *Overhead of recursive calls and returns*

Tabulated

- *Dynamic Programming*
- $T(n) = O(n)$
- *Space* = $O(n)$
- *No such stash overhead*

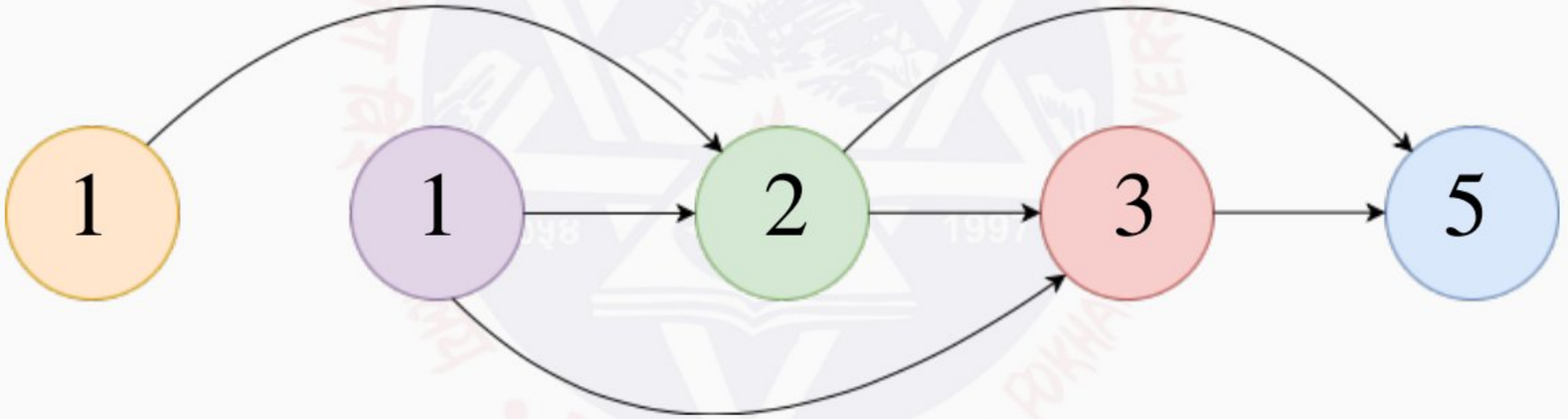
Which one do you prefer? Why?



CamScanner

fibo(n)

- Sub-problems dependency DAG



CamScanner

Resource Allocation using Dynamic Programming

- DP is a powerful optimization technique
- Idea of DP (you already know)
  - Break problems into subproblems,
  - Solve each subproblem once and store the result
  - Reuse sub-solutions to avoid redundant computations
- DP is mostly used in **resource allocation** problems
  - to allocate limited resources among competing demands to maximize or minimize a certain objective.



CamScanner

Resource Allocation using Dynamic Programming

- **Budget allocation**
  - *Problem*: Given a budget and multiple projects, each with a cost and return, now allocate the budget to maximize total return.
- **Resources Scheduling**
  - *Problem*: A company wants to allocate its workforce optimally across multiple tasks over time to minimize costs or maximize efficiency
- **Network bandwidth allocation**
  - *Problem*: Given multiple users requesting bandwidth and a total available bandwidth limit, allocate bandwidth to maximize total user satisfaction.
- **Production planning**
  - *Problem*: A manufacturer need to allocate production resources over time to minimize costs while meeting demand.



CamScanner

0/1 Knapsack Problem

- **Given:**
  - A set of n objects and a knapsack with capacity, W
  - Each object x_i $(i = 1,2,\ldots,n)$, has
    - a positive weight w_i
    - a positive value v_i.
- **Constraint:**
  - The knapsack carry a weight not exceeding W.

  Mathematically, $\sum_{i=1}^{n} x_i w_i \leq W$



CamScanner

0/1 Knapsack Problem

- **Object Selection**: Objects are either included (1) or excluded (0). So it it named as 0/1 Knapsack.

$$x_i = \begin{cases} 1 & if\ x_i \text{ is included} \\ 0 & otherwise \end{cases}$$

- **Objective**: To fill the knapsack in a way that maximizes the value of the included objects satisfying the capacity constraint.

Mathematically, maximize $\sum_{i=1}^{n} x_i v_i$



CamScanner

0/1 Knapsack using Tabulated DP

- Create a 2D array $V[i][w]$ for tabulation
- The maximum value achievable for the first i items and a knapsack capacity of w can be defined as

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & otherwise \text{ [item } i \text{ is included]} \end{cases}$$

-

CamScanner

0/1 Knapsack using Tabulated DP

- We are given 4 objects ($n = 4$) each with values $v_i = \{1,2,5,6\}$ and weights $w_i = \{2,3,4,5\}$ and knapsack capacity $W = 8$
- Goal: to maximize the total value without exceeding the knapsack's capacity $W = 8$

v_i	1	2	5	6
w_i	2	3	4	5

CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> we use Dynamic programming (Bottom-up)

>>

v_i	1	2	5	6
w_i	2	3	4	5



CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> we use Dynamic programming (Bottom-up)

>> use $V[i][w]$ represents max value achievable with first i items and knapsack capacity w.

>>

v_i	1	2	5	6
w_i	2	3	4	5



CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> we use Dynamic programming (Bottom-up)

>> use $V[i][w]$ represents max value achievable with first i items and knapsack capacity w.

>> the table $V[i][w]$ stores intermediate result to avoid redundant computations

v_i	1	2	5	6
w_i	2	3	4	5



CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> create (n+1) X (W +1) array $V[5][9]$ to store values $V[i][w]$ where

>>

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

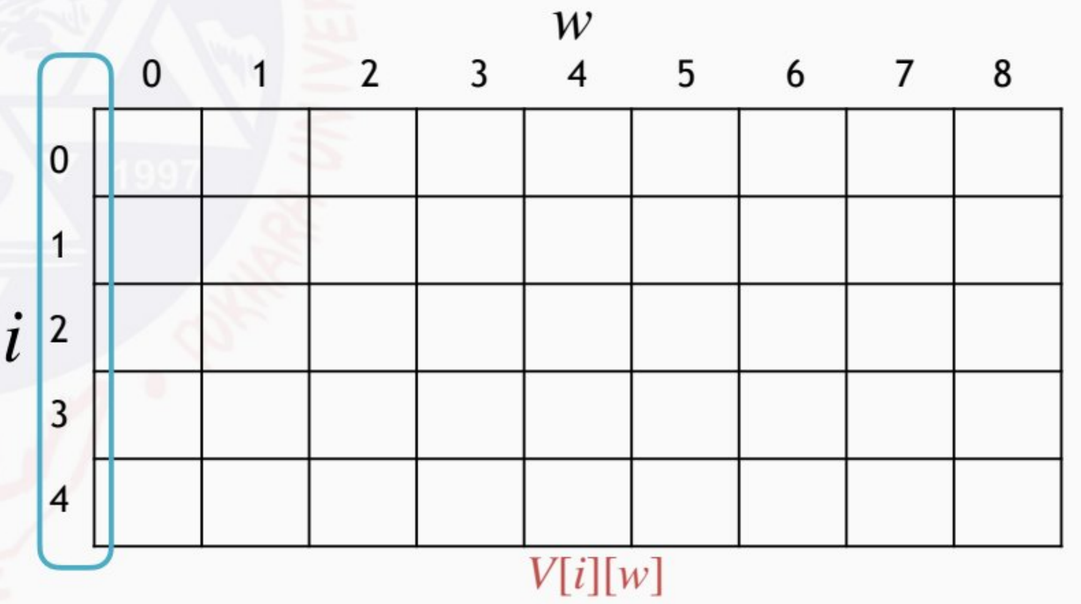
- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> create (n+1) X (W +1) array $V[5][9]$ to store values $V[i][w]$ where

>> Row: item i = 0 to 4

>>

v_i	1	2	5	6
w_i	2	3	4	5



CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

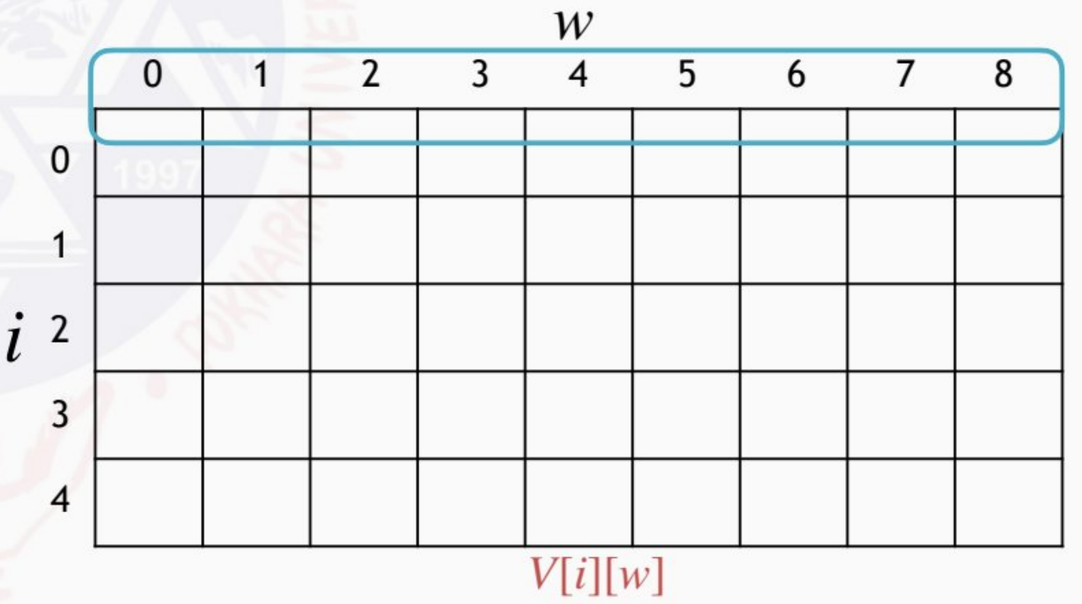
>> create (n+1) X (W +1) array $V[5][9]$ to store values $V[i][w]$ where

>> Row: item i = 0 to 4

>> Column: wight w = 0 to 8

>>

v_i	1	2	5	6
w_i	2	3	4	5



CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> Since item 0 ($i = 0$) does not exists,

Apply $V[0][w] = 0 \quad \forall w$

>>

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1									
2									
3									
4									

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> Since item 0 ($i = 0$) does not exists,

Apply $V[0][w] = 0 \quad \forall w$

>> Since no item has 0 capacity

Apply $V[i][0] = 0 \quad \forall i$

>>

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & otherwise \text{ [item } i \text{ is included]} \end{cases}$$

>> Now we fill the table row by row

>>

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 1$ ($w = 2$ and $v = 1$)

>> for all $w < 2$ cannot include item 1 (since wt. of item 1 is 2).

So, V[1][w] = 0 $\forall w < 2$

>> In class check V[1][w] for all 0 < w < 2

>>

$i = 1$

v_i	1	2	5	6
w_i	2	3	4	5

w

i	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0							
2	0								
3	0								
4	0								

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 1$ ($w = 2$ and $v = 1$)

>> for all $w >= 2$, can include item 1

So, $\forall w >= 2$

V[1][w] = max(V[0][w], 1 + V[0][w-2])

V[1][2] = max(V[0][2], 1 + V[0][2-2])

= max(0, 1) = 1 $\forall w >= 2$

>> In class check V[1][w] for all w = 2, 3, 4, …, 8.

$i = 1$

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0								
3	0								
4	0								

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & otherwise \text{ [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 2$ ($w = 3$ and $v = 2$)

>> for all $w < 3$ cannot include item 2.

So, V[2][w] = V[1][w] $\forall w < 3$

>> In class check V[2][w] for all w = 1, 2

>>

$i = 2$

v_i	1	2	5	6
w_i	2	3	4	5

w

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1						
3	0								
4	0								

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 2$ ($w = 3$ and $v = 2$)

>> for all $w >= 3$ can include item 2.

So, $\forall w >= 3$

V[2][w] = max(V[1][w], 2 + V[1][w-3])

V[2][3] = max(V[1][3], 2 + V[1][3-3]) =

= max(0, 2) = 2 for $w = 3$

>>

$i = 2$

v_i	1	2	5	6
w_i	2	3	4	5

w

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0								
4	0								

$V[i][w]$



CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 2$ ($w = 3$ and $v = 2$)

>> for all $w >= 3$ can include item 2.

So, $\forall w >= 3$

V[2][w] = max(V[1][w], 2 + V[1][w-3])

V[2][4] = max(V[1][4], 2 + V[1][4-3]) =

= max(1, 2+0) = 2 for $w = 4$

>>

$i = 2$

v_i	1	2	5	6
w_i	2	3	4	5

w

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0								
4	0								

$V[i][w]$

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 2$ ($w = 3$ and $v = 2$)

>> for all $w >= 3$ can include item 2.

So, $\forall w >= 3$

V[2][w] = max(V[1][w], 2 + V[1][w-3])

V[2][5] = max(V[1][5], 2 + V[1][5-3]) =

= max(1, 2+1) = 3

>> In class check V[1][w] for all w = 4, 5, …, 8.

$i = 2$

v_i	1	2	5	6
w_i	2	3	4	5

$V[i][w]$

	w 0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0								
4	0								

CamScanner

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & otherwise \text{ [item } i \text{ is included]} \end{cases}$$

>> Process item $i = 3$ ($w = 4$ and $v = 5$)

>>

$i = 3$

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0								

$V[i][w]$

0/1 Knapsack using Tabulated DP

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if } w_i > w \text{ [item } i \text{ is excluded]} \\ max(V[i-1][w], v_i + V[i-1][w-w_i]) & \text{otherwise [item } i \text{ is included]} \end{cases}$$

>> Similarly,

Process item $i = 4$ ($w = 5$ and $v = 6$)

$i = 4$

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

- ***Goal***: to maximize the total value without exceeding the knapsack's capacity $W = 8$

>> Now the maximized value is found at V[n][W]

i.e. at V[4][8]

v_i	1	2	5	6
w_i	2	3	4	5

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

$V[i][w]$

CamScanner

0/1 Knapsack using Tabulated DP

- Implementation of our example in C++

```
int knapsack_tabulation(int W, vector<int>& weights, vector<int>& values, int n) {
    vector<vector<int>> V(n + 1, vector<int>(W + 1, 0));
    // fill the table V in bottom-up manner
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weights[i - 1] > w) {
                V[i][w] = V[i - 1][w]; // Exclude item
            } else {
                V[i][w] = max(V[i - 1][w], values[i - 1] + V[i - 1][w - weights[i - 1]]);
            }
        }
    }
    return V[n][W]; // Maximum profit
}
```



CamScanner

Analysis of 0/1 Knapsack using Tabulated DP

- Implementation of our example in C++

```
int knapsack_tabulation(int W, vector<int>& weights, vector<int>& values, int n) {
    vector<vector<int>> V(n + 1, vector<int>(W + 1, 0));
    // fill the table V in bottom-up manner
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weights[i - 1] > w) {
                V[i][w] = V[i - 1][w]; // Exclude item
            } else {
                V[i][w] = max(V[i - 1][w], values[i - 1] + V[i - 1][w - weights[i - 1]]);
            }
        }
    }
    return V[n][W]; // Maximum profit
}
```

n

W

Time Complexity: $O(nW)$

Space Complexity: $O(nW)$

CamScanner

Engineering Applications of DP

- **Network bandwidth allocation:**
  - ***Problem***:
    - You have N users, each requesting a certain amount of bandwidth.
    - Each user has a **satisfaction score** associated with receiving their requested bandwidth
    - The total available **bandwidth capacity** is **B**.
  - ***Goal***: **maximize total satisfaction** while ensuring the total allocated bandwidth **does not exceed B.**



CamScanner

Engineering Applications of DP

- **Network bandwidth allocation:**
  - ***Problem simplification***:

    Items → Users

    Weight → Requested Bandwidth w_iw i

    Value → Satisfaction Score v_i

    Knapsack Capacity → Total Available Bandwidth B

    Object Selection: $x_i = \begin{cases} 1 & if\ x_i \text{ is included} \\ 0 & otherwise \end{cases}$

  - ***Goal***: **maximize** $\sum_{i=1}^{N} x_i v_i$, **with Constraint:** $\sum_{i=1}^{N} x_i w_i \leq B$

CamScanner

Engineering Applications of DP

- Implementation of Network bandwidth allocation in C++ (0/1 knapsack)

```cpp
int maxSatisfaction(int B, vector<int>& bandwidths, vector<int>& satisfaction, int N) {
    vector<vector<int>> V(N + 1, vector<int>(B + 1, 0));

    // fill the table V in bottom-up manner
    for (int i = 1; i <= N; i++) {
        for (int b = 0; b <= B; b++) {
            if (bandwidths[i - 1] > b) {
                V[i][b] = V[i - 1][b];  // Exclude user i
            } else {
                // Include or Exclude user i
                V[i][b] = max(V[i - 1][b], satisfaction[i - 1] + V[i - 1][b - bandwidths[i - 1]]);
            }
        }
    }
    return V[N][B];  // Maximum total satisfaction
}
```


CamScanner

Other Engineering Applications of DP

- **Budget allocation**
  - *Problem*: Given a budget and multiple projects, each with a cost and return, now allocate the budget to maximize total return.
- **Resources Scheduling**
  - *Problem*: A company wants to allocate its workforce optimally across multiple tasks over time to minimize costs or maximize efficiency
- **Production planning**
  - *Problem*: A manufacturer need to allocate production resources over time to minimize costs while meeting demand.



CamScanner

1. Introduction to Huffman coding
2. Huffman coding- data compression in embedded systems

GREEDY ALGORITHM

CamScanner

Greedy Algorithms

- An approach or a design for solving a problem
- Solve optimization problems
  - Which demands either to minimize or maximize their results
- Optimization problems = ???
  - ***For example:*** I want to travel from Pkr to Ktm with minimum cost and in time less that 1 hour.



CamScanner

Greedy Algorithms

- Suppose I have a Problem

$$P$$



CamScanner

Greedy Algorithms

- to travel from Pokhara to Kathmandu

$$P : Pkr \longrightarrow Ktm$$



CamScanner

Greedy Algorithms

- I can travel by car

$$P : Pkr \longrightarrow Ktm$$

Car

CamScanner

Greedy Algorithms

- I can travel by bus

$$P : Pkr \longrightarrow Ktm$$

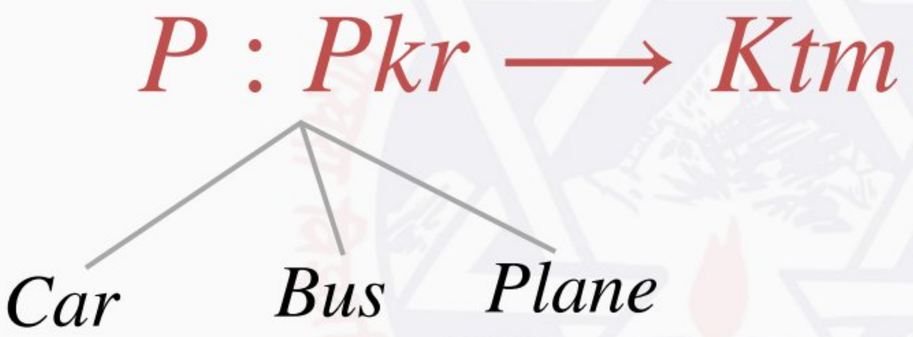
Car *Bus*



CamScanner

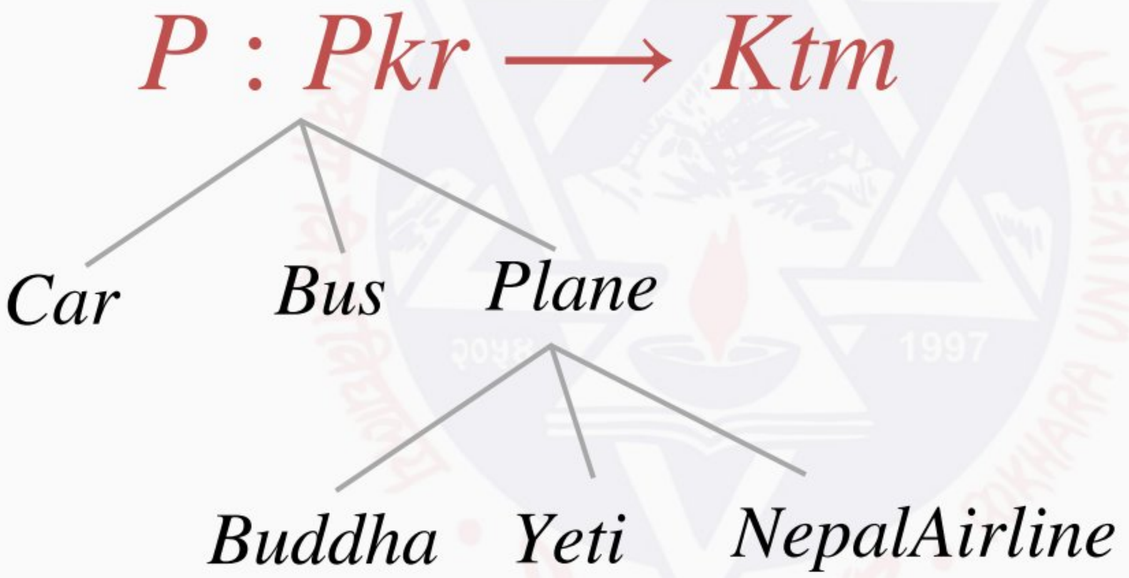
Greedy Algorithms

- I can travel by plane



Greedy Algorithms

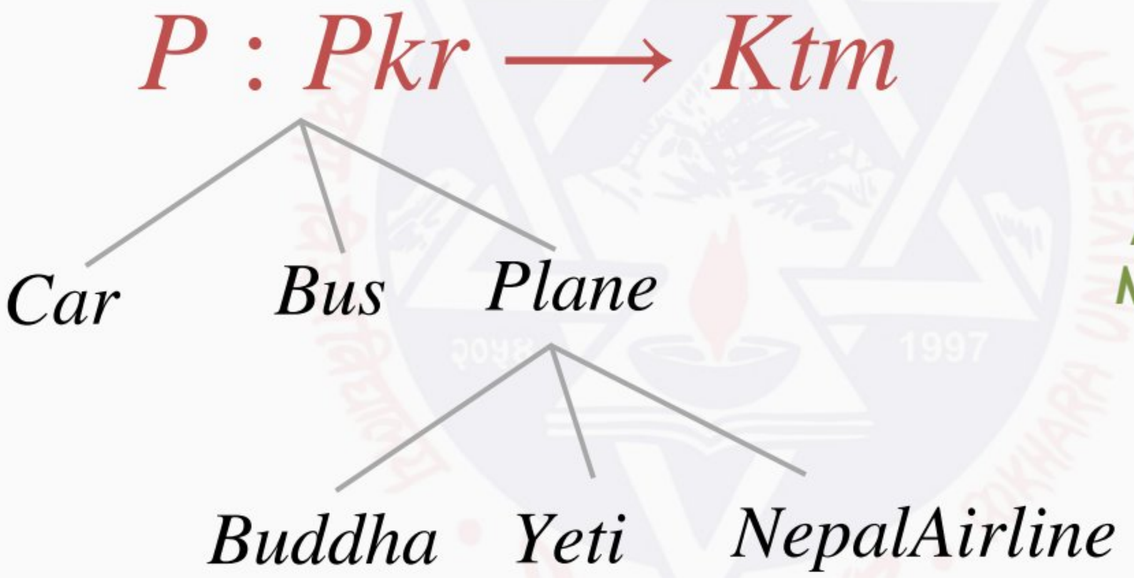
- I can travel by plane



CamScanner

Greedy Algorithms

- I can travel by plane

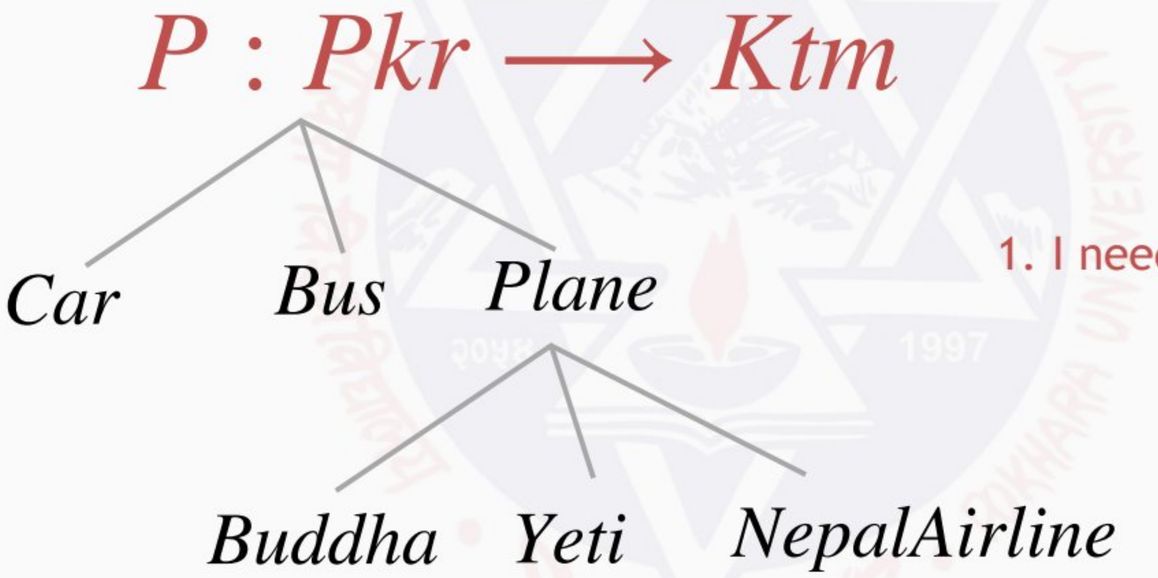


A problem can have
Multiple Solutions

CamScanner

Greedy Algorithms

- I can travel by plane

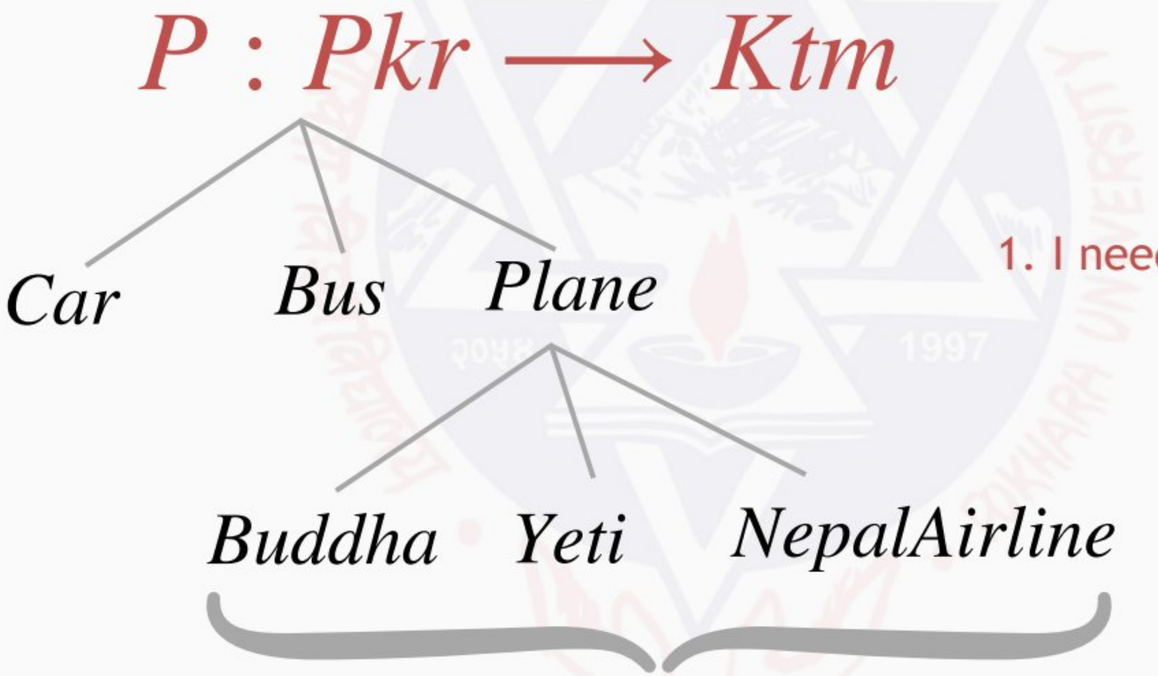


Constraints:
1. I need to reach Ktm within 1 hour

CamScanner

Greedy Algorithms

- I can travel by plane



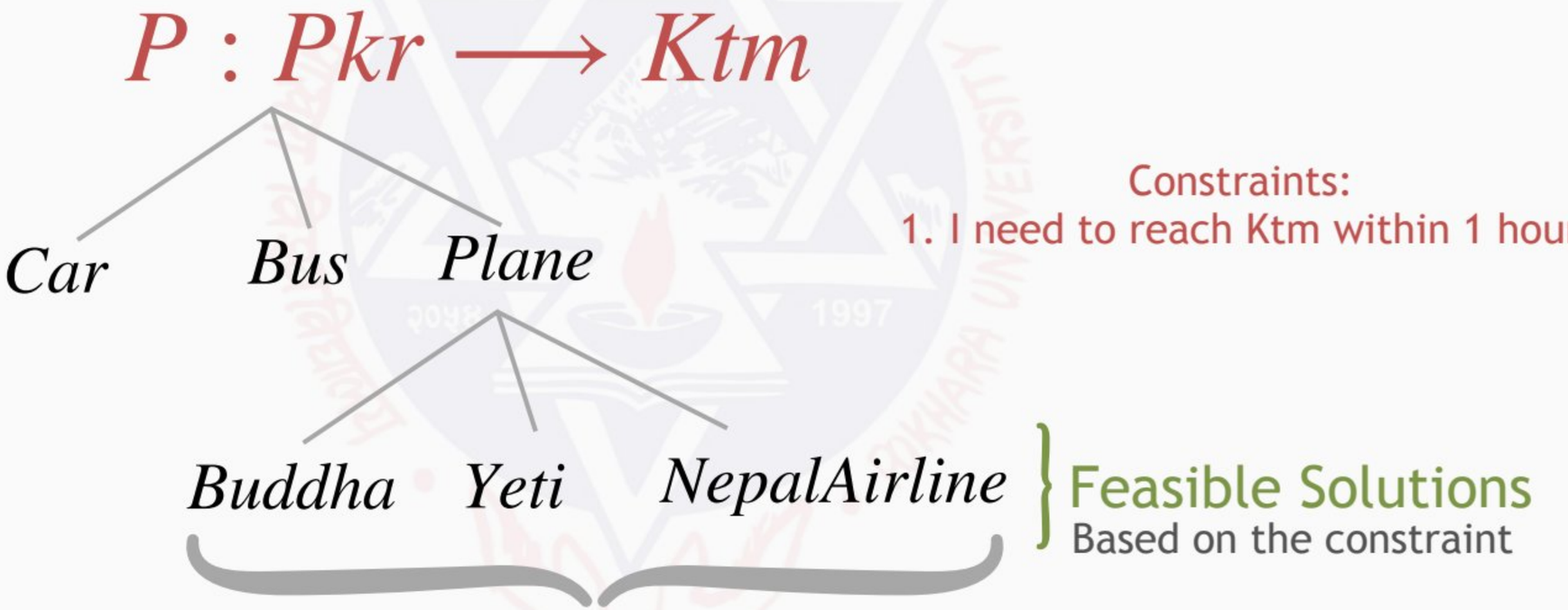
Constraints:
1. I need to reach Ktm within 1 hour

I am restricted to travel only by plane to meet constraint.

CamScanner

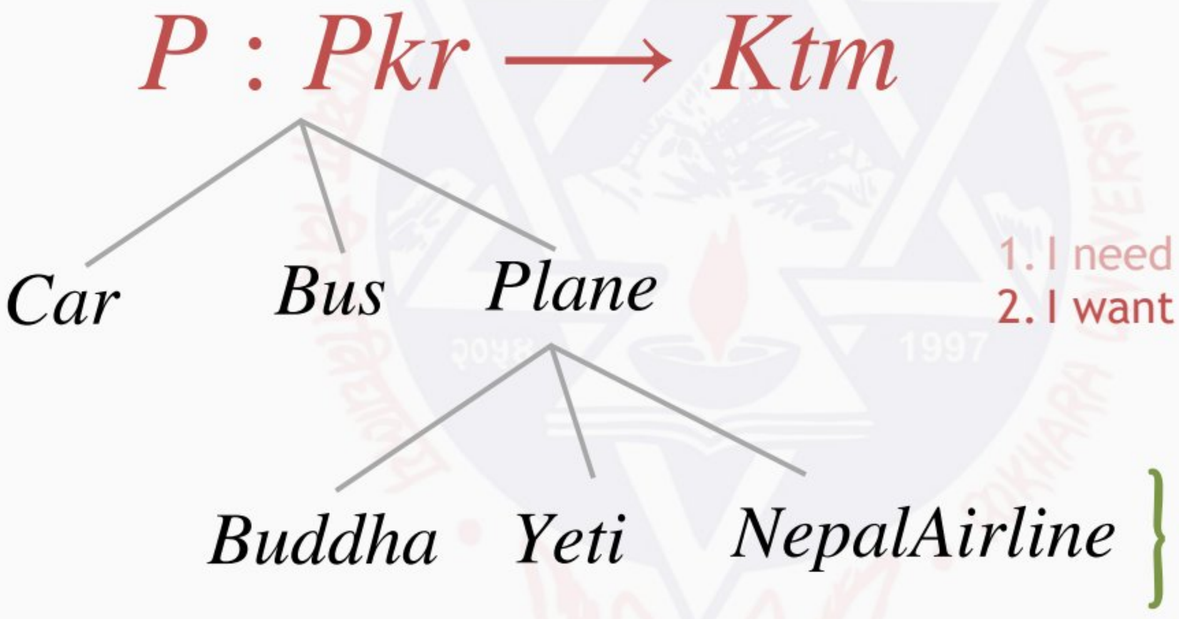
Greedy Algorithms

- I can travel by plane



Greedy Algorithms

- Suppose I want the cheaper travel (**Greedy Approach**).



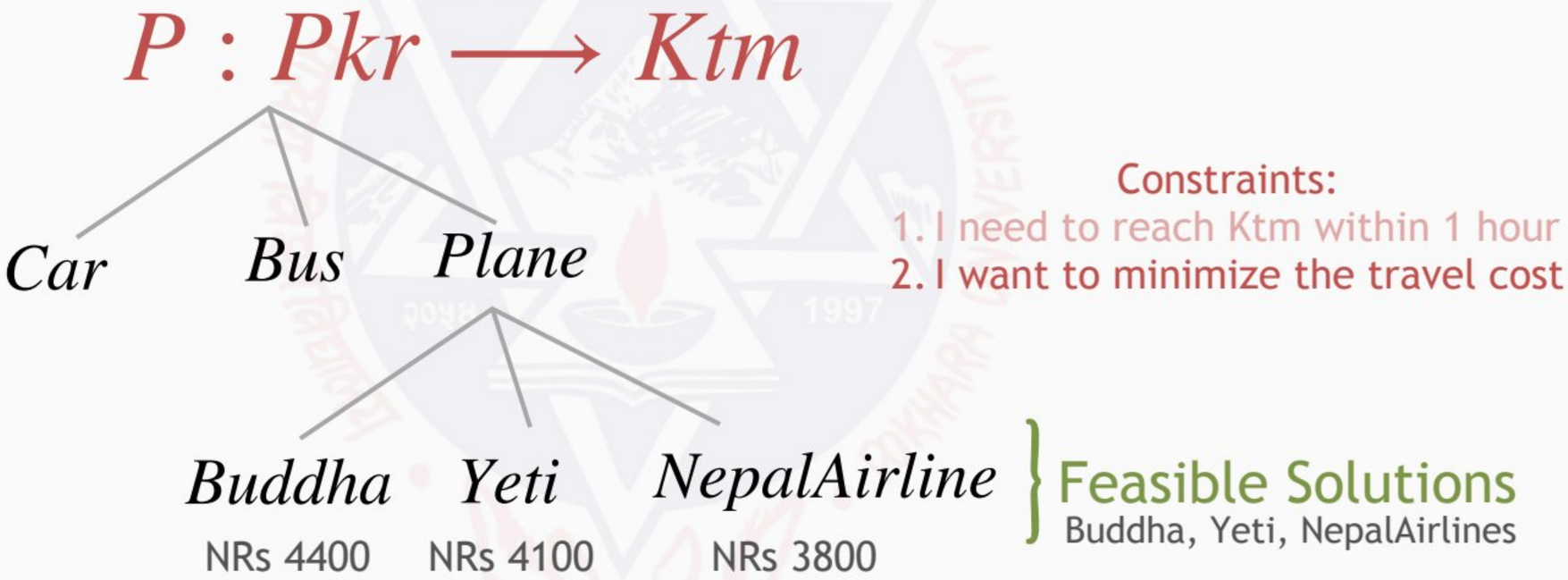
Constraints:
1. I need to reach Ktm within 1 hour
2. I want to minimize the travel cost

} Feasible Solutions
Based on the constraint

CamScanner

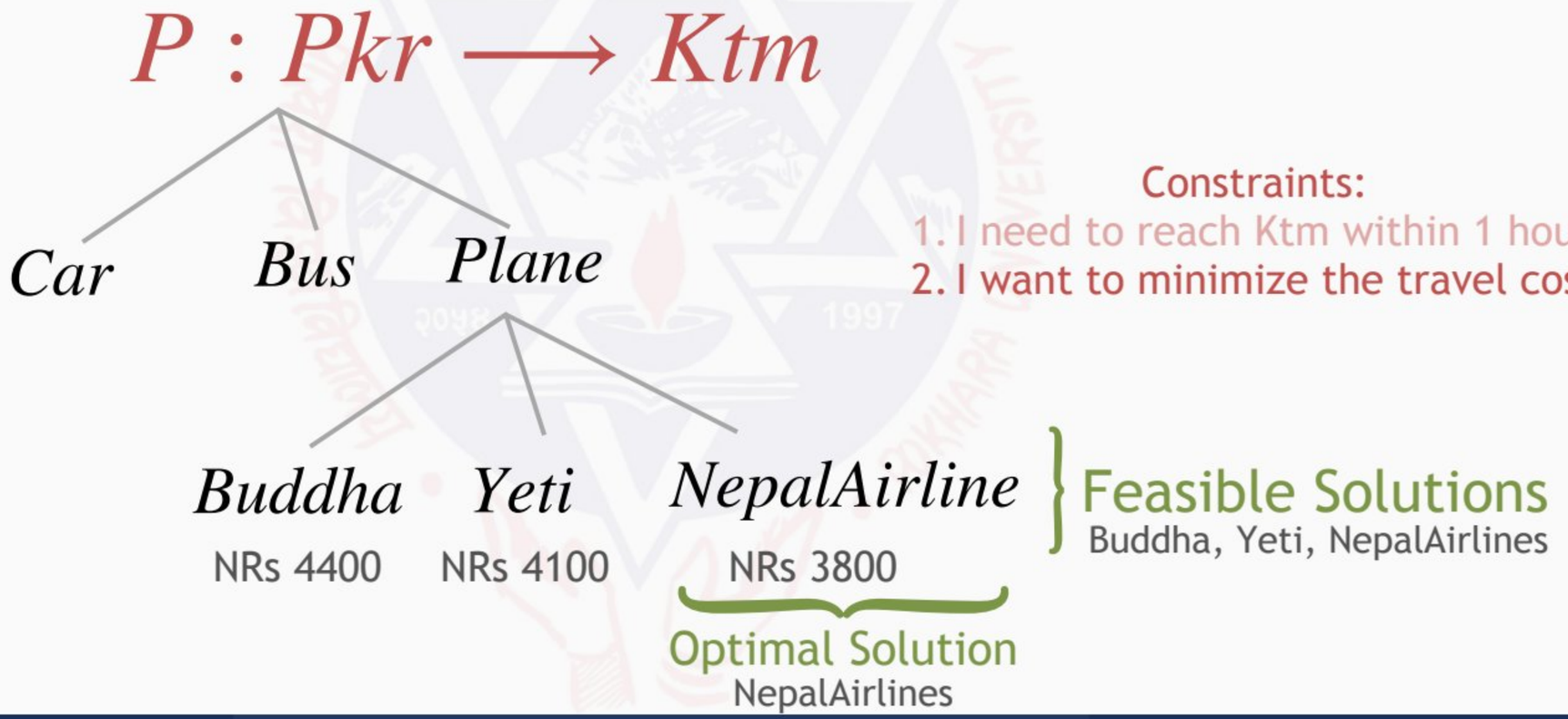
Greedy Algorithms

- Suppose the cost of different airlines



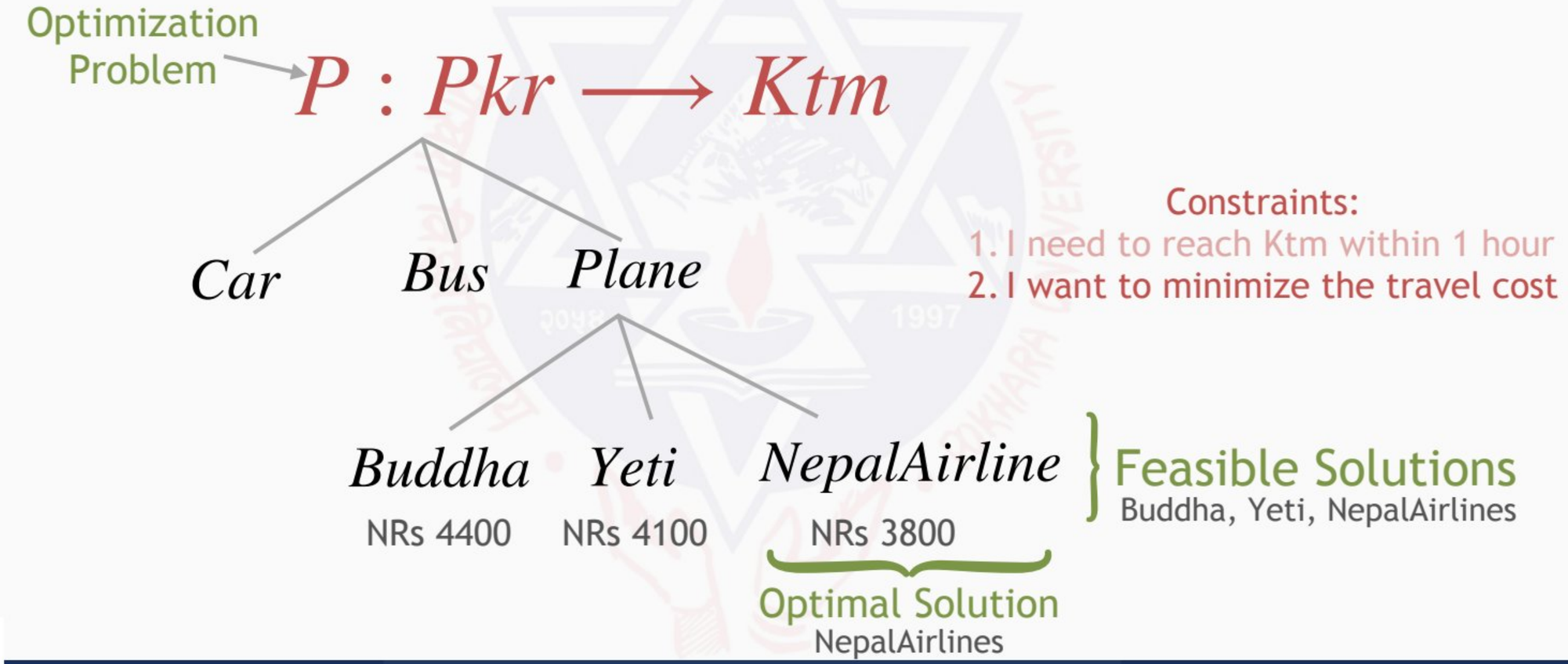
Greedy Algorithms

- Suppose the cost of different airlines



Greedy Algorithms

- So P is an optimization problem



Data Compression

- Fixed-length Coding
  - Example: ***ASCII coding***
  - takes seven bits
    - to generate the 128 (= 2^7) unique codes
  - Can represent
    - ASCII character set
- ***Problem*** of fixed-length coding
  - Takes more memory to store data
    - **how ??**



CamScanner

Data Compression

- E appears about 60 times more often than the letter Z.
- Given a string "AAAEEEELLXZ", calculate the space it takes to store it in memory.

Space = 77x3x7 + 120x4x7 + 42x2x7 + 4x1x7
+ 2x1x7
= 801x7 = 5607 bits

- Can you figure out the problem here?

Letter	Frequency	Letter	Frequency
A	77	N	67
B	17	O	67
C	32	P	20
D	42	Q	5
E	120	R	59
F	24	S	67
G	17	T	85
H	50	U	37
I	76	V	12
J	4	W	22
K	7	X	4
L	42	Y	22
M	24	Z	2

CamScanner

Data Compression

- What if you apply the ***idea***:
  - ***assign*** the fewer-bits code for frequently used letters.

    Let E is coded by 1, A by 0, L by 01 X by 101, Z by 0001

    Space for "AAAEEEELLXZ"

    = 77x3x1 + 120x4x1 + 42x2x2+ 4x1x3 + 2x1x4

    = 899 bits

- Size is *compressed* by 6.23
- This is called ***variable-length coding***.
  - Example: Huffman Coding

Letter	Frequency	Letter	Frequency
A	77	N	67
B	17	O	67
C	32	P	20
D	42	Q	5
E	120	R	59
F	24	S	67
G	17	T	85
H	50	U	37
I	76	V	12
J	4	W	22
K	7	X	4
L	42	Y	22
M	24	Z	2

CamScanner

Huffman Coding

- Provides ***lossless*** data compression
- ***Idea***:
  - reduces the size of data by encoding frequently occurring symbols with shorter bit representations and less frequent symbols with longer ones.
- ***Why it is Greedy***
  - *Try to use minimize number of bits to encode frequently occurring symbols*
- ***Widely used in embedded systems***
  - Storage efficiency
  - Data transmission efficiency



CamScanner

Embedded System

- A specialized computing system designed to perform dedicated functions within a larger system.
- ***Optimized for specific task***
  - Designed for a single function or a set of few related functions
- ***Processing***: MPU (Microprocessor) or MCU (Microcontroller)
- ***Memory***: ROM and RAM
- ***Power Supply:*** battery-powered or an external power source

CamScanner

Embedded System

- ***Operating System***:
  - Bare Metal (No OS): Direct firmware execution
  - Real-Time OS
- Disadvantage
  - ***Limited Resources***
    - ***Limited*** memory, power, and processing
- ∴ **Huffman-coding:** for ***efficient*** data storage and data transmission



CamScanner

Huffman Coding

- Developed by David Huffman in 1951
- It uses full binary tree to generate Huffman code for each letter
  - The tree is called- ***Huffman Tree***
- ***Goal***: to build a tree with the minimum weighted path.
- Each leaf of corresponds to a letter
- Define the weighted path length of a leaf to be its weight times its depth
- A letter with high weight (frequency) should have low depth
  - letter with low height (frequency) is pushed deeper in the tree



CamScanner

Huffman Coding

```
HuffmanTree(frequencyTable){
1.    Create a priority_queue (min-heap) Pq
2.    Insert all characters with their frequencies into Pq
3.    while(priority_queue.size() > 1)
4.        Remove: two nodes N1 and N2
5.        Create a new_node
6.        new_node.frequency = N1.frequency + N2.frequency
7.        new_node.left = N! And new_node.right = N2
8.        Insert new_node back into the priority queue Pq
9.    // after "while" loop finishes,
10.   // the remaining node in the Pq is the root of the Huffman Tree
11.   // Store character codes
12.   Create a HuffmanDictionary huffmanCode = {}
13.   generateHuffmanCodes(Pq, "", & huffmanCode)
11.   Return huffmanCode
```



CamScanner

Huffman Coding

```
generateHuffmanCodes(node, currentNode, &huffmanCode){
1. If node is a leaf
2.     huffmanCode[node.character] = currentCode
3.     Return
3.  If node has a left child
4.       generateHuffmanCodes(node.left, currentCode + "0")
5.  If node has a right child
6.       generateHuffmanCodes(node.right, currentCode + "1")
}
```



CamScanner

Huffman Coding: Example

- Find the Huffman coding for the following characters:

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1



CamScanner

Huffman Coding: Example

- Sort the letter in ascending order of their frequencies

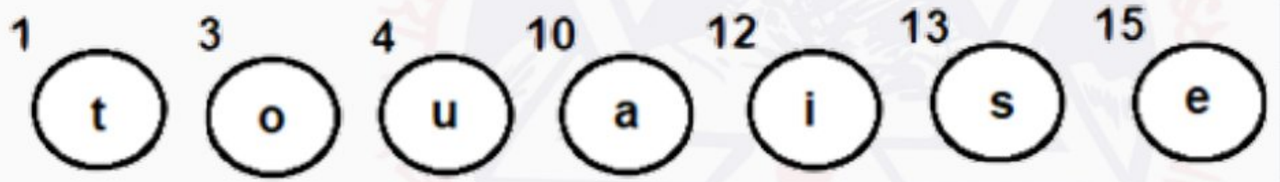


Fig 1: Leaf nodes for each character

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1

CamScanner

Huffman Coding: Example

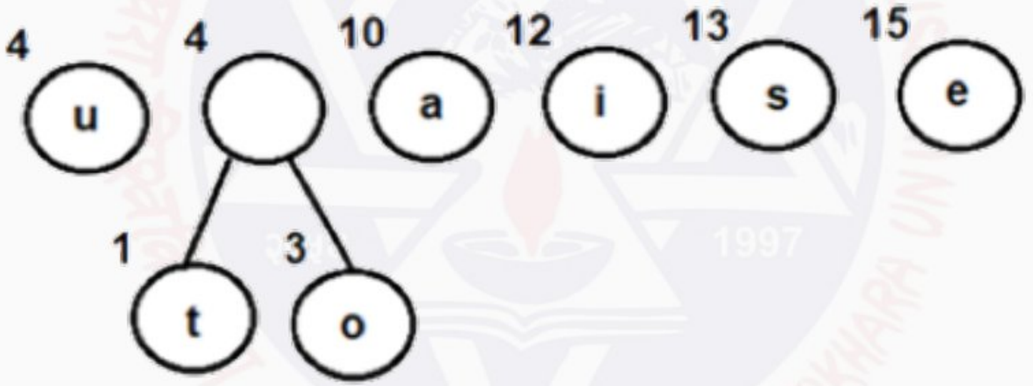


Fig 2: Combining nodes o and t

CamScanner

Huffman Coding: Example

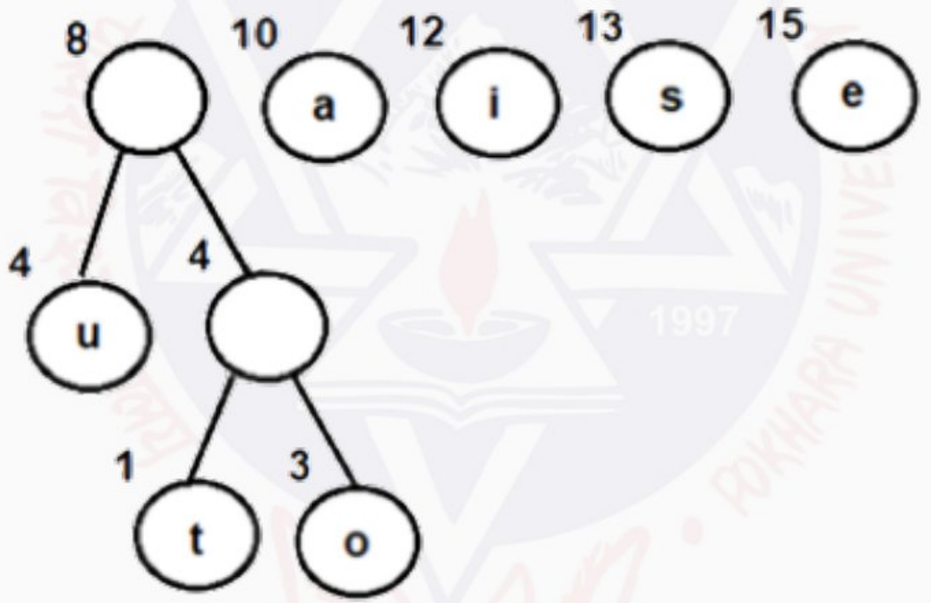
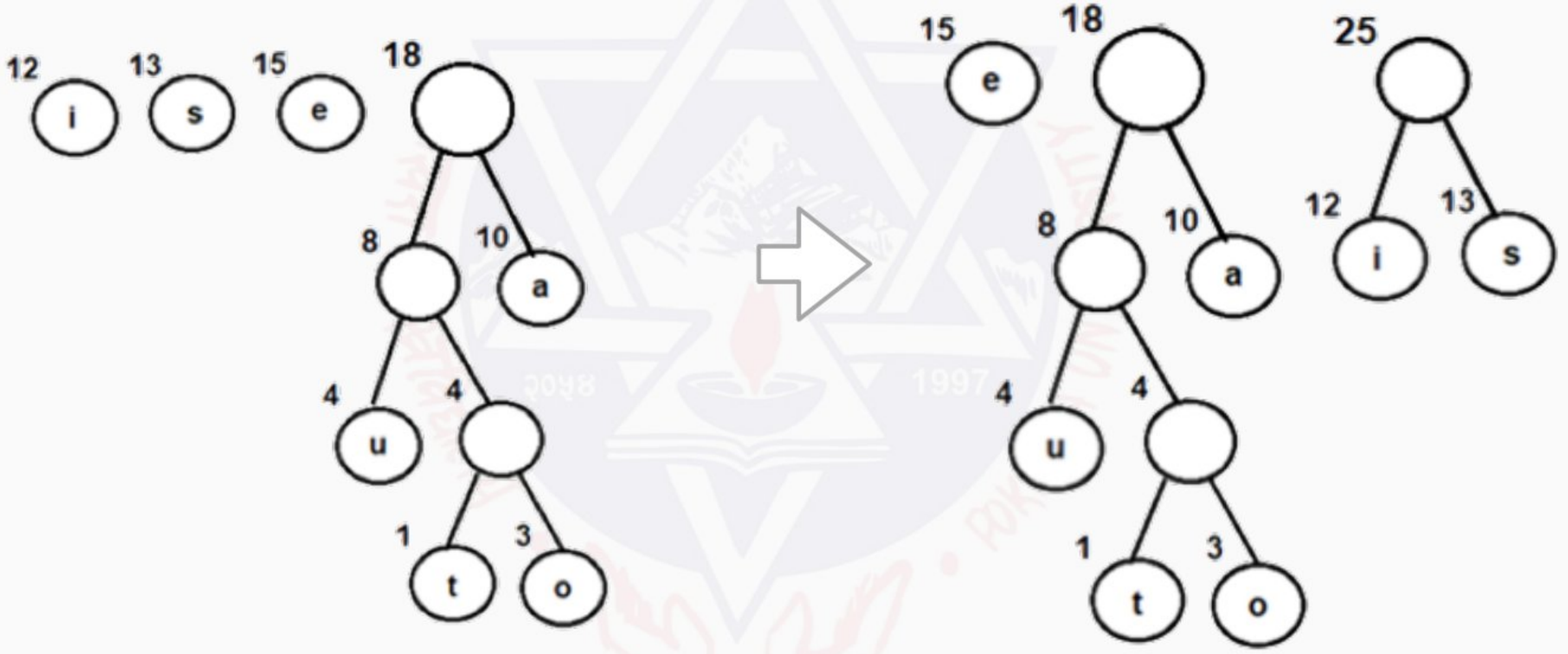
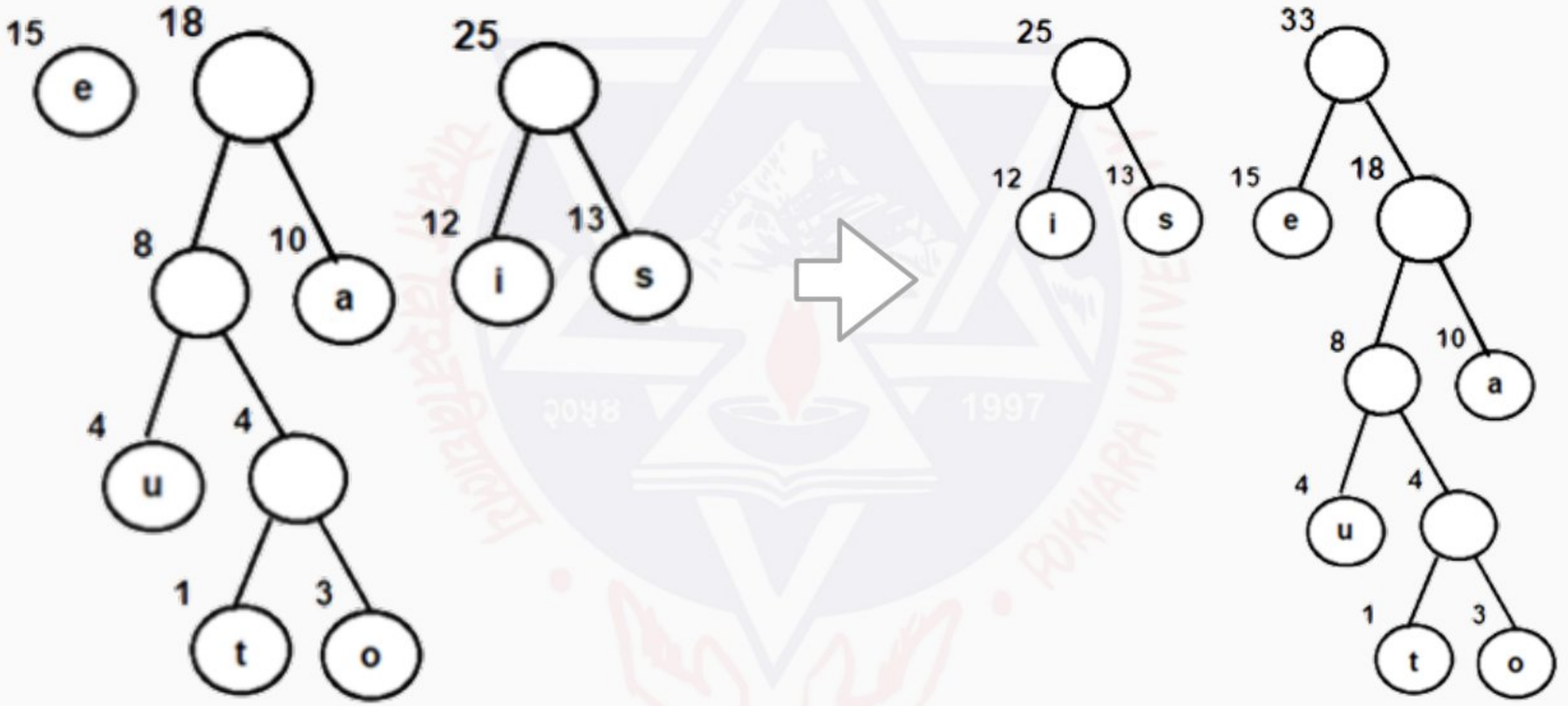


Fig 3: Combining node u with an internal node having 4 as frequency

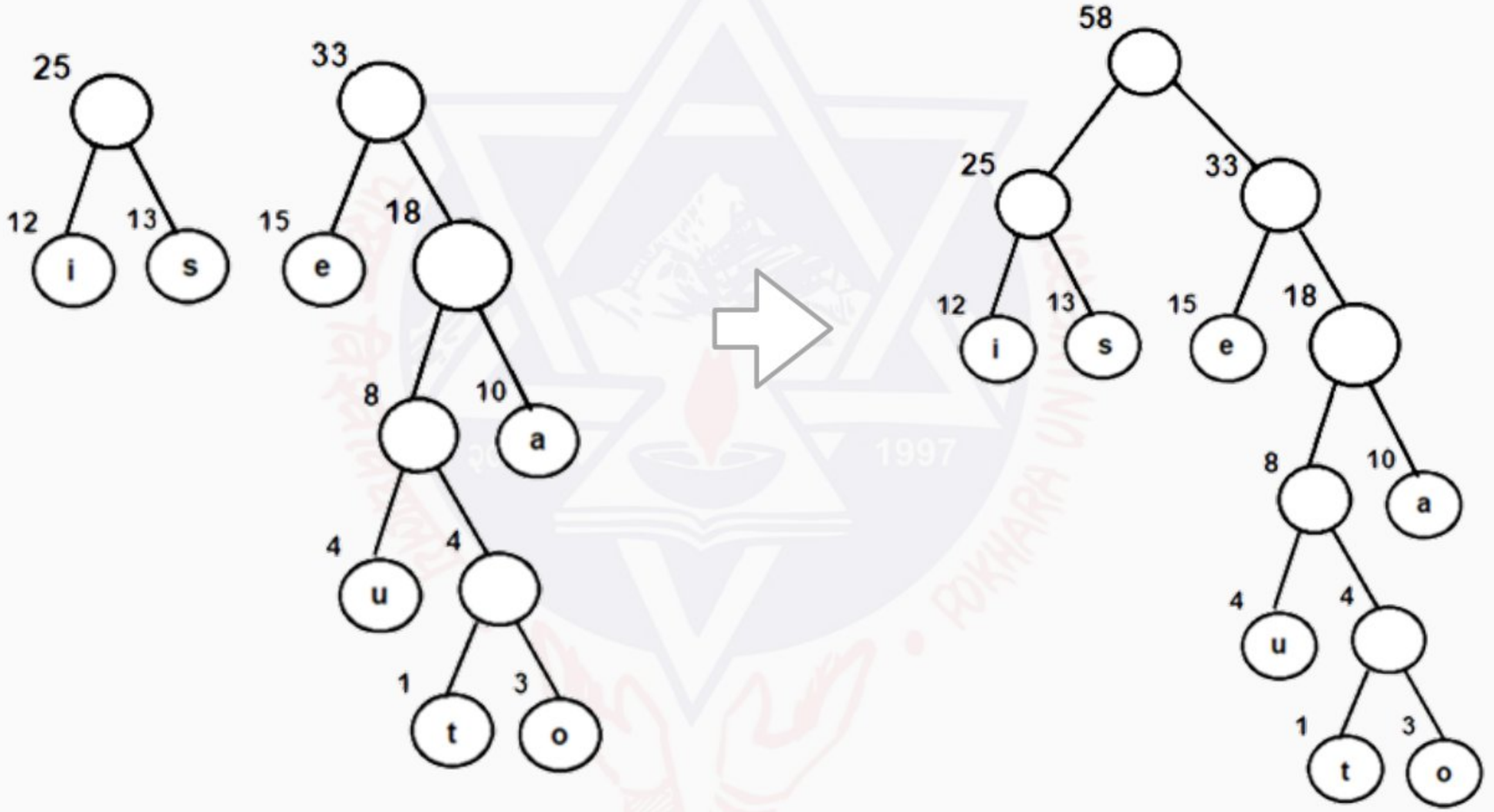
Huffman Coding: Example



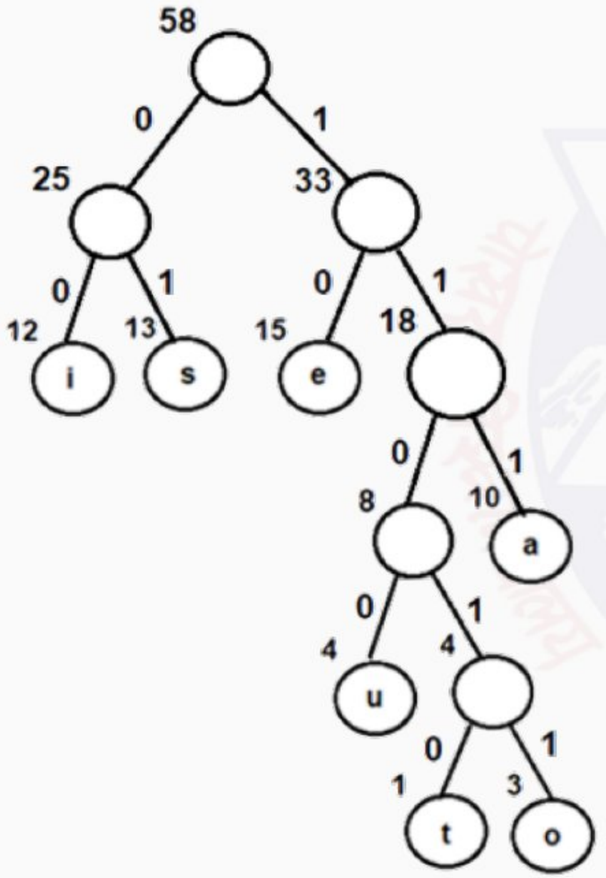
Huffman Coding: Example



Huffman Coding: Example



Huffman Coding: Example



(a)

Characters	Huffman Code
i	00
s	01
e	10
u	1100
t	11010
o	11011
a	111

(b)

Fig 8: Assigning binary codes to Huffman tree (a) Huffman tree after assigning 0 to the left child and 1 to the right child (b) generated variable length Huffman coding for each character

CamScanner

Implementation of Huffman Coding in C++

Demo in Class

CamScanner

Implementation of Huffman Coding in C++

- Add the following two function in the program
  - Encoding Function
  - Decoding function
- Take any string of those letters and observe the encoding and decoding results



CamScanner

Static Huffman Coding for Embedded System

- Optimization of Huffman Coding is essential
  - It is because Embedded systems have
    - Limited RAM
    - Limited CPU computing power
    - Limited power supply (in battery powered ES)
- Optimized Huffman Coding
  - Static Huffman Coding



CamScanner

Static Huffman Coding for Embedded System

- The Huffman tree is precomputed and stored in Flash or ROM.
- No need to build the tree at runtime
  - reduces CPU and RAM usage.
- Use lookup tables
  - reduces computation during encoding and decoding
- Good for fixed datasets
  - e.g., sensor data, predefined messages
- Minimize memory allocation
  - avoid dynamic structures like priority queues (require more memory)
- Optimize for low-power consumption
  - use bitwise operations (executes in single clock. So very fast)



CamScanner

Implementation: Static Huffman Coding

```cpp
#include <iostream>
#include <unordered_map>
#include <bitset>

using namespace std;

// Precomputed Static Huffman Codes for a Known Dataset
unordered_map<char, string> staticHuffmanCodes = {
    {'A', "00"}, {'B', "01"}, {'C', "100"}, {'D', "101"},
    {'E', "110"}, {'F', "1110"}, {'G', "1111"}
};

// Reverse lookup table for decoding
unordered_map<string, char> staticHuffmanDecoding = {
    {"00", 'A'}, {"01", 'B'}, {"100", 'C'}, {"101", 'D'},
    {"110", 'E'}, {"1110", 'F'}, {"1111", 'G'}
};
```



CamScanner

Implementation: Static Huffman Coding

```
string encode(const string &text) {
    string encodedText = "";
    for (char ch : text) {
        encodedText += staticHuffmanCodes[ch];  // Lookup precomputed code
    }
    return encodedText;
}
string decode(const string &encodedText) {
    string decodedText = "";
    string buffer = "";
    for (char bit : encodedText) {
        buffer += bit;  // Collect bits
        if (staticHuffmanDecoding.find(buffer) != staticHuffmanDecoding.end()) {
            decodedText += staticHuffmanDecoding[buffer];  // Decode character
            buffer = "";  // Reset buffer
        }
    }
    return decodedText;
}
```



CamScanner

Implementation: Static Huffman Coding

```cpp
int main() {
    string text = "ABCD";

    string encodedText = encode(text);
    cout << "Original Text: " << text << endl;
    cout << "Encoded Text: " << encodedText << endl;

    string decodedText = decode(encodedText);
    cout << "Decoded Text: " << decodedText << endl;

    return 0;
}
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS ··· Code

```
[Running] cd "/Users/udayadhungana/dynamic_programming/" && g++ -std=c
++17 static_huffman.cpp -o static_huffman && "/Users/udayadhungana/
dynamic_programming/"static_huffman
Original Text: ABCD
Encoded Text: 0001100101
Decoded Text: ABCD

[Done] exited with code=0 in 3.537 seconds
```


CamScanner



THANK YOU

CamScanner